

JAWAHARLAL NEHRU TECHNOLOGICAL UNIVERSITY

GURAJADA VIZIANAGARAM

VIZIANAGARAM – 533003

ANDHRA PRADESH

WASTECARE

By

NUTHULAPATI SOWMYA SREE

(Regd. No. 24VV1A1243)

Under the supervision of

MRS.MADHUMITHA CHANDA

MRS.M. GEETHA MADHURI YADAV

Department of Information Technology

Vizianagaram – 533003

JNTU-GURAJADA VIZIANAGARAM
COLLEGE OF ENGINEERING VIZIANAGARAM (A)
VIZIANAGARAM

Regd.No : 24VV1A1243

CERTIFICATE

This is to certify that this is a bonafide record of practical work done by **NUTHULAPATI SOWMYASREE** of IIIrd B.Tech Ist Semester Class in User Interface Design using Flutter Lab during the year 2026-27.

No.of Tasks Completed and Certified: 11

Lecture In-Charge **Head of The Department**

Date:

DEPARTMENT OF INFORMATION TECHNOLOGY
JNTU-GURAJADA VIZIANAGARAM
COLLEGE OF ENGINEERING VIZIANAGARAM (A)
VIZIANAGARAM

Website:www.jntugvcev.edu.in

Subject Name: USER INTERFACE DESIGN USING FLUTTER Subject Code: R233112ES01

Academic Year: 2026 **Regulation: R23**

COURSE OUTCOMES

NBA Subject Code	Course Outcomes	
R233112ES01	CO1	Learns to Implement Flutter Widgets and Layouts.
	CO2	Understands Responsive UI Design and with Navigation in Flutter.
	CO3	Knowledge on Widgets and customize widgets for specific UI elements, Themes
	CO4	Understand to include animation apart from fetching data.

CO-PO Mapping

Mapping of Course Outcomes (COs) with Program Outcomes (POs)

Course Outcomes		Program Outcomes (POs)														
		PO 1	PO 2	PO 3	PO 4	PO 5	PO 6	PO 7	PO 8	PO 9	PO 10	PO 11	PO 12	PSO 1	PSO 2	PSO 3
R233112ES01	CO1	3	2	3	2	3	1	1	1	2	2	1	2	3	3	2
	CO2	3	3	3	2	3	1	2	2	2	3	2	2	3	3	2
	CO3	3	2	3	2	3	2	1	1	2	2	1	2	3	2	2
	CO4	3	3	3	3	3	2	1	2	3	3	2	3	3	3	3

Enter correlation levels 1,2 and 3 as defined below:

1: Slight (Low) 2: Moderate (Medium) 3: Substantial (High) If there is no correlation, put "-"

Signature of the Course Instructor

CONTENTS

S.No.	Date	Experiment Name	Page No.	Marks	Signature
1(a)		Installation of Flutter and Dart SDK	1		
1(b)		Dart Programming Basics	2		
2(a)		Explore Flutter Widgets	3		
2(b)		Layouts Using Row, Column, and Stack	5		
3(a)		Responsive User Interface	7		
3(b)		Media Queries and Breakpoints	9		
4(a)		Navigation Using Navigator	11		
4(b)		Navigation Using Named Routes	13		
5(a)		Stateful and Stateless Widgets	15		
5(b)		State Management Using setState	17		
6(a)		Creating Custom Widgets	18		
6(b)		Themes and Custom Styles	20		
7(a)		Creating Forms with Input Fields	21		
7(b)		Form Validation and Error Handling	23		
8(a)		Implementing Animations	24		
8(b)		Different Animation Types	25		
9(a)		Connecting to a REST API	26		
9(b)		Displaying Fetched API Data	27		

10(a)		Unit Testing in Flutter	28		
10(b)		Debugging a Flutter Application	29		
11		Case Study: wastecare (Road Damage Reporting System)	30		

Experiment 1(a): Install Flutter and Dart SDK

Aim

To install flutter and dart sdk using examples relevant to the WasteCare project.

Description

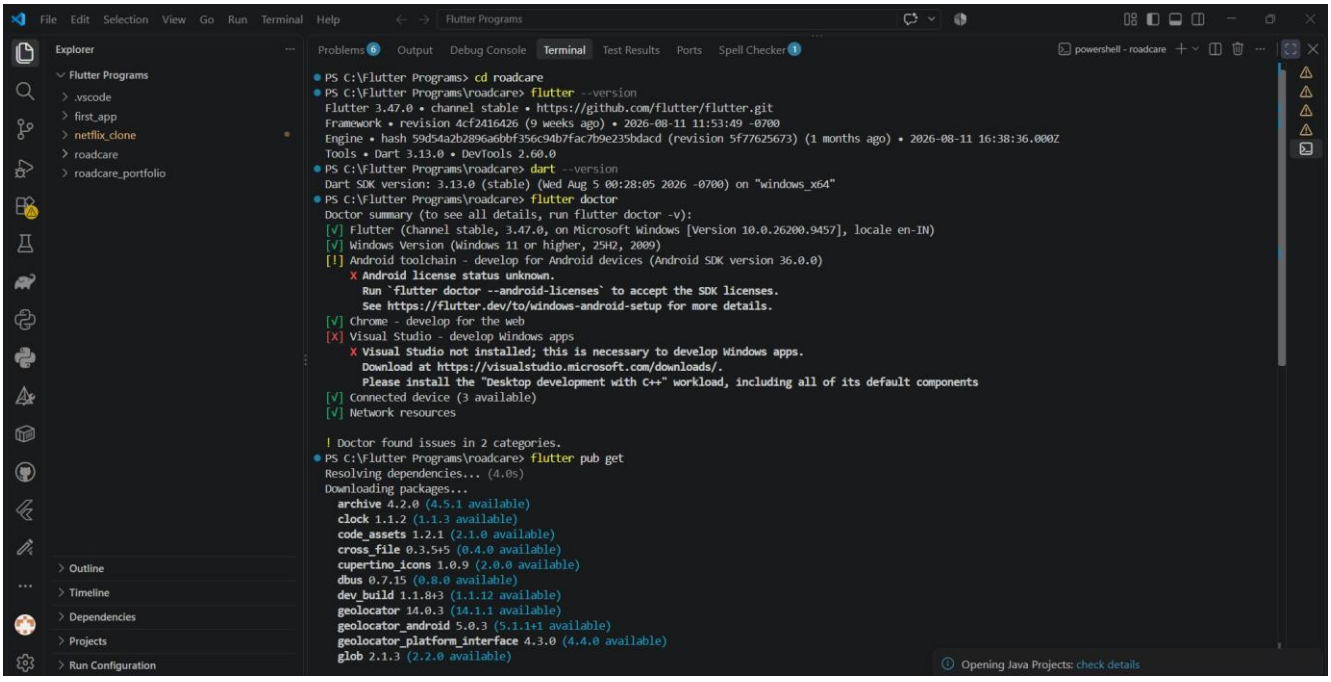
Flutter is a cross-platform UI framework used to build the WasteCare interface from a shared Dart codebase. The Dart SDK provides the language runtime and development tools required by Flutter. The Flutter command-line tools help verify the installation, download project dependencies, identify available devices and launch the application. Android and Chrome are useful target platforms for checking the same project on a mobile device and a browser. The terminal output should be captured after running these commands on the development computer.

Code

```
flutter --version
dart --version
flutter doctor
flutter pub get
 flutter run -d chrome
flutter devices
```

Output

Flutter is a cross-platform UI toolkit. Dart is the language used to implement WasteCare. The project is run from VS Code and tested on Android and Chrome.



Experiment 1(b): Write a simple Dart program to understand language basics

Aim

To write a simple dart program to understand language basics using examples relevant to the WasteCare project.

Description

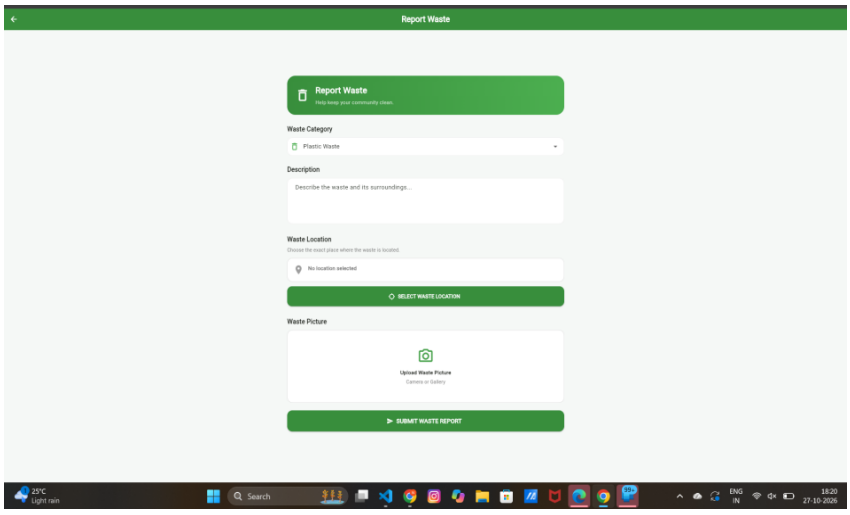
Dart is the programming language used to describe widgets, application logic and data models in Flutter. Variables store values such as report titles and selected severity, while lists can hold available severity categories. Functions group reusable operations and string interpolation combines labels with variable values. Conditions and loops help process form data and display report information. This small program demonstrates Dart basics using WasteCare-related values and its printed output should be captured from the Dart terminal.

Code

```
void main () {
final projectName = ' WasteCare ';
 final categories = [
 ' Plastic Waste ' ,
 ' Paper Waste ' ,
 ' Metal Waste ' ,
 ' Glass Waste ' ,
 ' Food Waste ' ,
 ' Electronic Waste '
 ];
 print ( ' Project : $projectName ') ;
 for ( final category in categories ) {
 print ( ' Category : $category ') ;
 }
 }
```

Output

Dart variables, functions, strings and interpolation form the basis of wastecare model and validation code



Experiment 2(a): Explore Flutter widgets (Text, Image, Container, etc.)

Aim

To explore flutter widgets (text, image, container, etc.) using examples relevant to the WasteCare project.

Description

Flutter interfaces are built by composing widgets rather than by writing separate native layouts for each platform. Text widgets display headings, labels and report descriptions; Container controls padding, background and decoration; Image presents selected road-damage evidence; and buttons trigger actions. Cards and icons make the dashboard easier to scan,
while consistent widget styling improves readability. wasteCare combines these widgets to show report information and guide users through common actions. Capture the actual dashboard screen as evidence for this experiment.

Code

```
Container (
 padding : const EdgeInsets . all (16) ,
 decoration : BoxDecoration (
 color : const Color (0 xFFF1F8F3 ) ,
 borderRadius : BorderRadius . circular (16) ,
 ) ,
 child : const Column (
 crossAxisAlignment : CrossAxisAlignment . start ,
 children : [
 Text (
 ' WasteCare ' ,
 style : TextStyle (
 fontSize : 22 ,
 fontWeight : FontWeight . bold ,

) ,
 ) ,
 Text ( ' Waste Management and Reporting System ') ,
' WasteCare ' ,
 style : TextStyle (
 fontSize : 22 ,
 fontWeight : FontWeight . bold ,

) ,
 ) ,

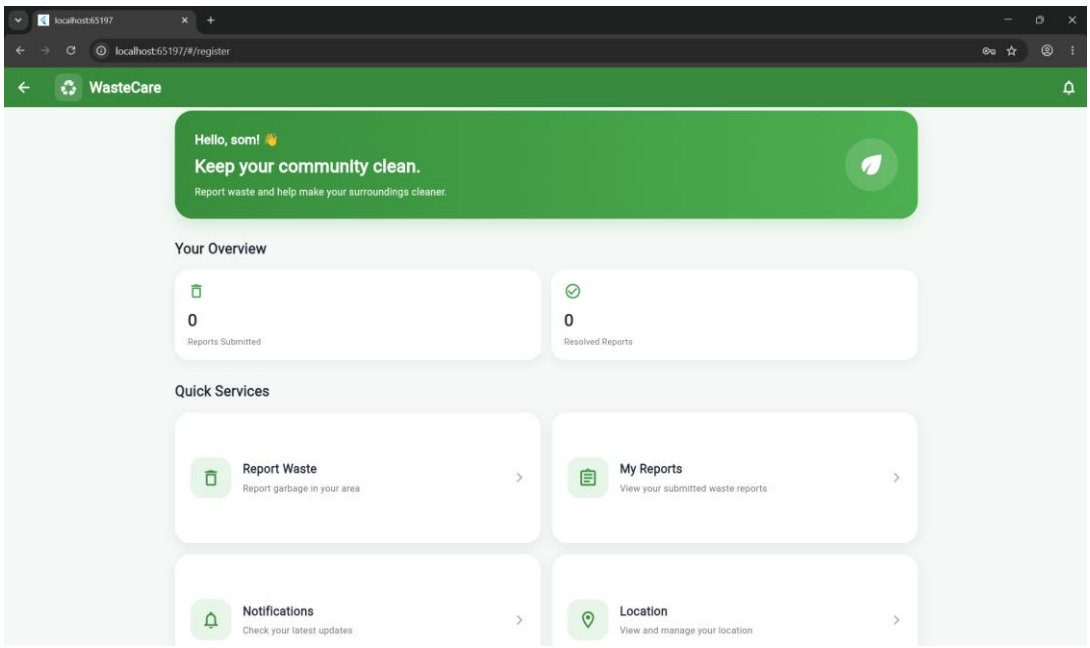
 ] ,
```

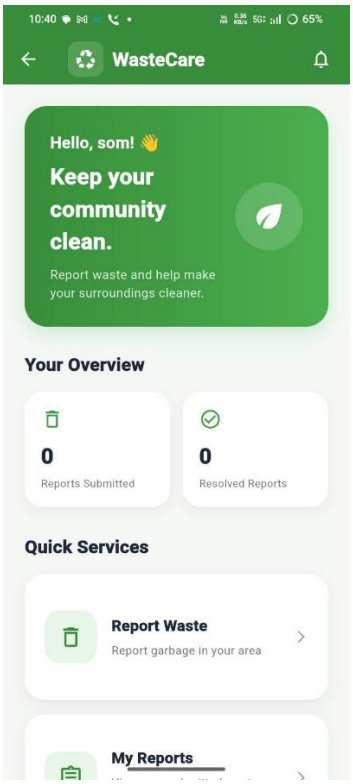
```
' WasteCare ' ,
style : TextStyle (
fontSize : 22 ,
fontWeight : FontWeight . bold ,
) ,
) ,

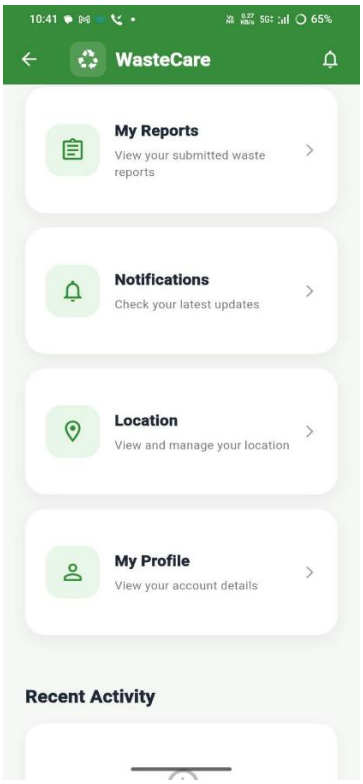
] ,
```

Output

WasteCare screens are composed from widgets. Text shows report information, Container decorates sections, Image displays evidence and buttons trigger actions.




10:40
65%
WasteCare
Hello, som! 👋
Keep your community clean.
Report waste and help make your surroundings cleaner.
Your Overview
0
Reports Submitted
0
Resolved Reports
Quick Services
Report Waste
Report garbage in your area
My Reports


10:41
65%
WasteCare
My Reports
View your submitted waste reports
Notifications
Check your latest updates
Location
View and manage your location
My Profile
View your account details
Recent Activity

Experiment 2(b): Implement layouts using Row, Column and Stack

Aim

To implement layouts using row, column and stack using examples relevant to the RoadCare project.

Description

Row places widgets horizontally, while Column arranges them vertically; both are commonly used in dashboards and report forms. Stack positions widgets on top of one another, which is useful for placing a severity badge over an image or a status label on a card. Layouts should be arranged so that labels remain readable and controls are easy to tap. The sample illustrates a layered image and severity chip, but asset names must be changed to match files that actually exist in the project. Capture the closest real WasteCare card or detail layout.

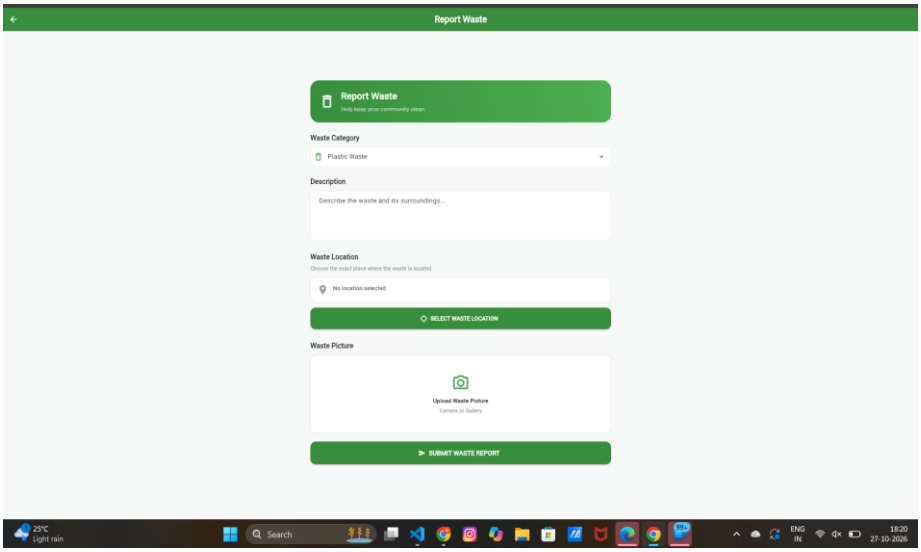
`code

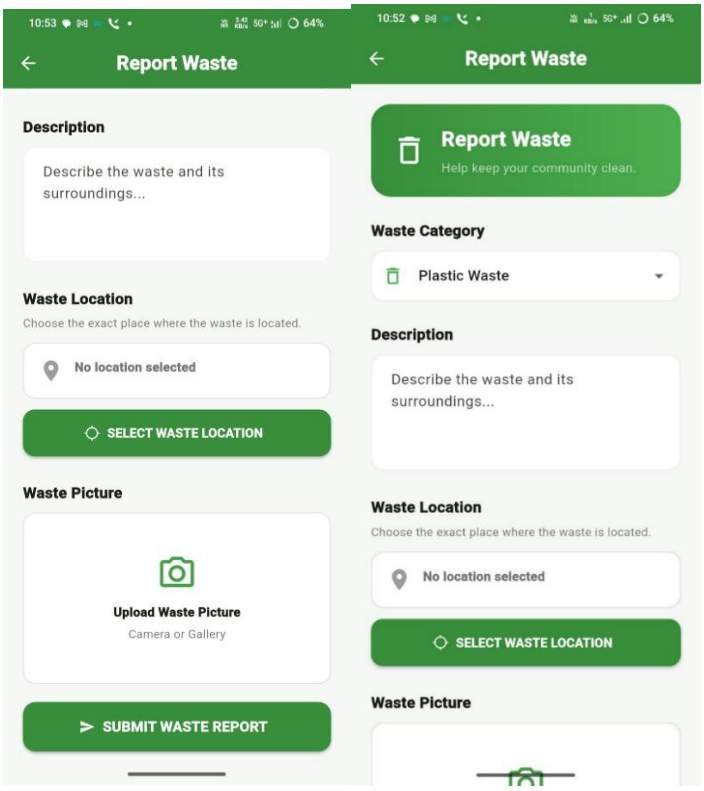
```
column (
 crossAxisAlignment :
   CrossAxisAlignment
   . start ,
 children : [
 const Text ( ' Waste
   Report ') ,
 const SizedBox (
   height : 12) ,
 Row (
 children : [
 const Icon ( Icons .
   delete_outline ) ,
 const SizedBox (
   width : 8) ,
 const Expanded (
 child : Text ( ' Plastic
   Waste ') ,
) ,
] ,
) ,

] ,
)
```

Output

Row and Column arrange dashboard elements. Stack can layer a badge or icon over a report image.





Experiment 3(a): Design a responsive UI for different screen sizes Aim

To design a responsive ui for different screen sizes using examples relevant to the WasteCare project.

Description

Responsive design makes an interface adapt to the space available on a phone, tablet or desktop browser. A narrow Android screen may need a single-column list, whereas a wide Chrome window can show several report cards side by side. Flutter can use LayoutBuilder to inspect the available constraints and choose a suitable layout. The breakpoint should be selected by testing real content rather than by assuming every device has the same width. Capture the same Care screen on Android and Chrome to demonstrate the resulting differences.

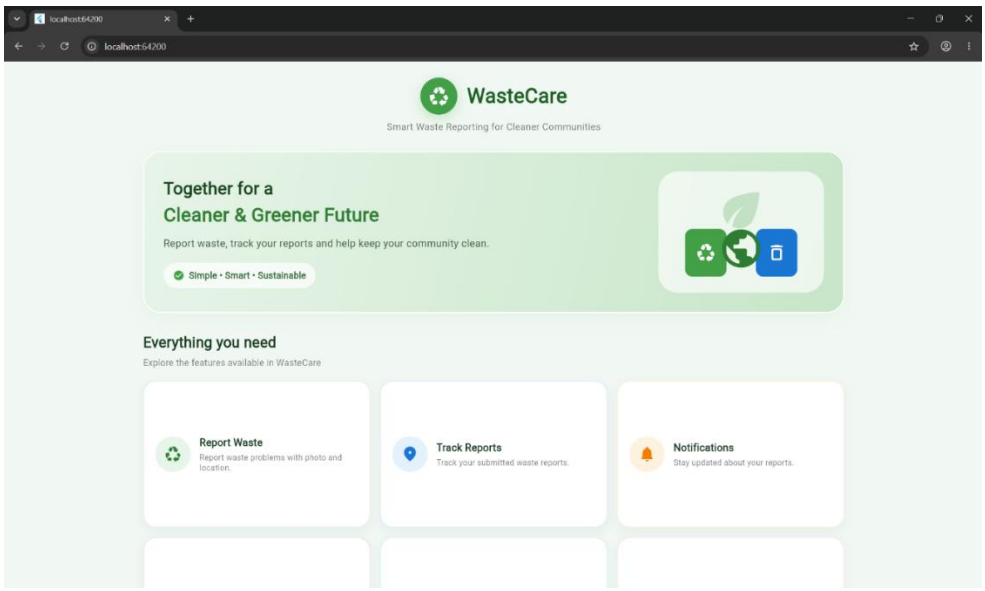
Code

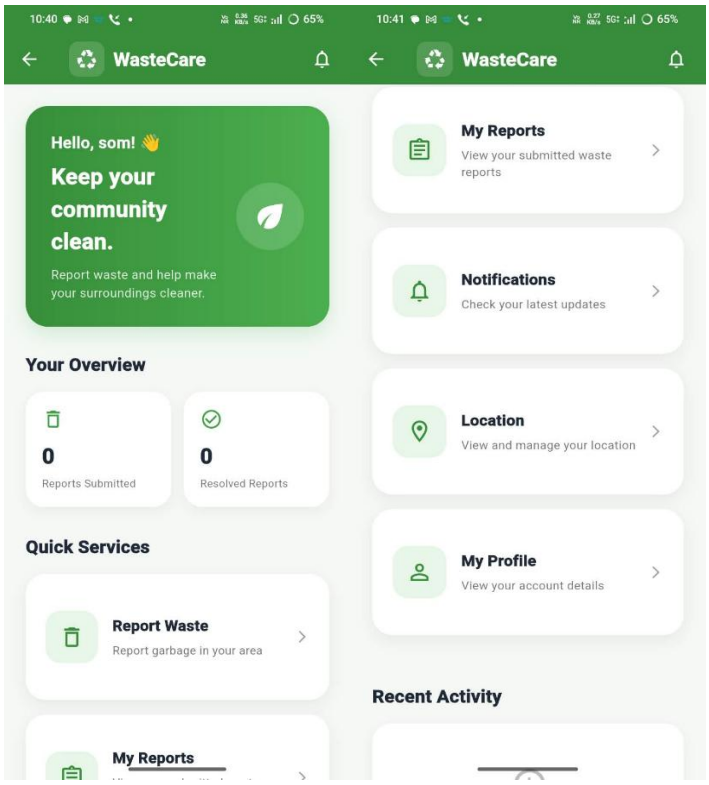
```
LayoutBuilder (
 builder : ( context , constraints ) {
 final columns =
 constraints . maxWidth > 700 ? 2 : 1;

 return GridView . count (
 crossAxisCount : columns ,
 children : reportCards ,
 ) ;
 } ,
}
```

Output

Responsive design keeps WasteCare usable on narrow Android screens and wider Chrome windows. Layout decisions should depend on available width.




10:40
65%
WasteCare
Hello, som! 👋
Keep your community clean.
Report waste and help make your surroundings cleaner.
Your Overview
0
Reports Submitted
0
Resolved Reports
Quick Services
Report Waste
Report garbage in your area
My Reports
10:41
65%
WasteCare
My Reports
View your submitted waste reports
Notifications
Check your latest updates
Location
View and manage your location
My Profile
View your account details
Recent Activity

Experiment 3(b): Implement media queries and breakpoints for responsiveness

Aim

To implement media queries and breakpoints for responsiveness using examples relevant to the WasteCare project.

Description

Flutter does not use CSS media queries in the same way as a web page; MediaQuery and LayoutBuilder provide screen dimensions and layout constraints. These values can be used to define breakpoints and change the number of columns, spacing or navigation presentation. A good responsive layout avoids clipped text, overflowing cards and controls that become too small on narrow screens. WasteCare report lists and dashboard cards are useful examples because they contain content that must remain readable across screen sizes. The screenshots should show the layout at both narrow and wide widths.

Code

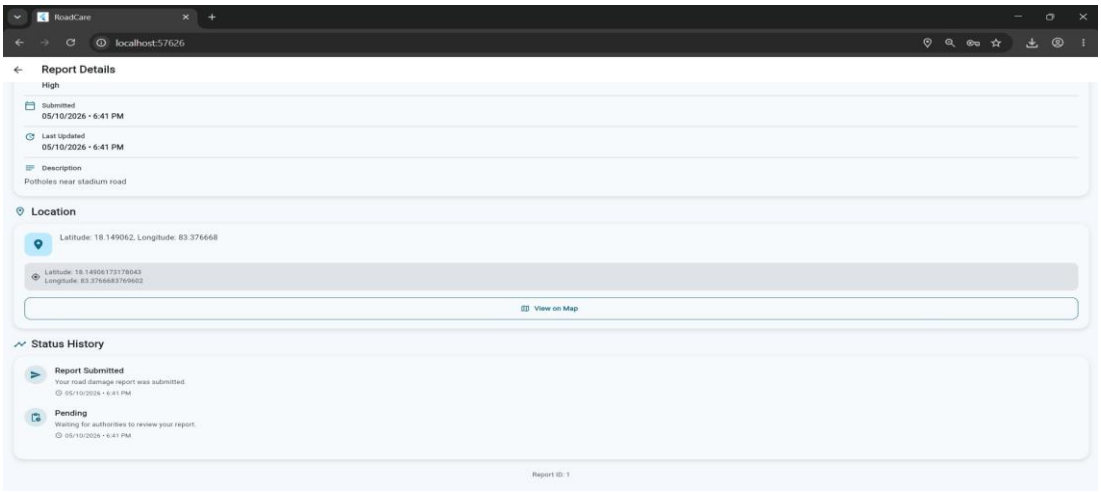
```
final width = MediaQuery . of ( context ) . size . width ;

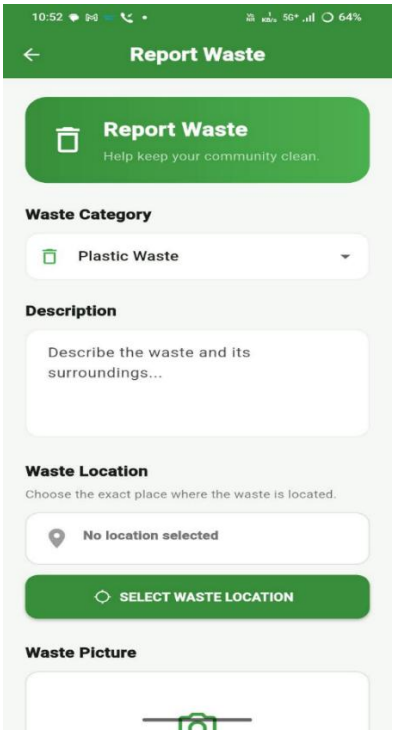
 LayoutBuilder (
 builder : ( context , constraints ) {
 final columnCount =
 constraints . maxWidth > 700 ? 2 : 1;

 return GridView . count (
 crossAxisCount : columnCount ,
 children : reportCards ,
 ) ;
 } ,
)
```

Output

Flutter does not use CSS media queries; MediaQuery and LayoutBuilder provide screen size and responsive breakpoints.





Experiment 4(a): Set up navigation between screens using Navigator

Aim

To set up navigation between screens using navigator using examples relevant to the WasteCare project.

Description

Navigator manages the route stack used to move between Flutter screens. Navigator.push opens a destination screen, and Navigator.pop returns to the previous screen when appropriate. In WasteCare, users may move from the dashboard to a report form and then to report details or history. Navigation should preserve necessary data and should not unexpectedly send an already

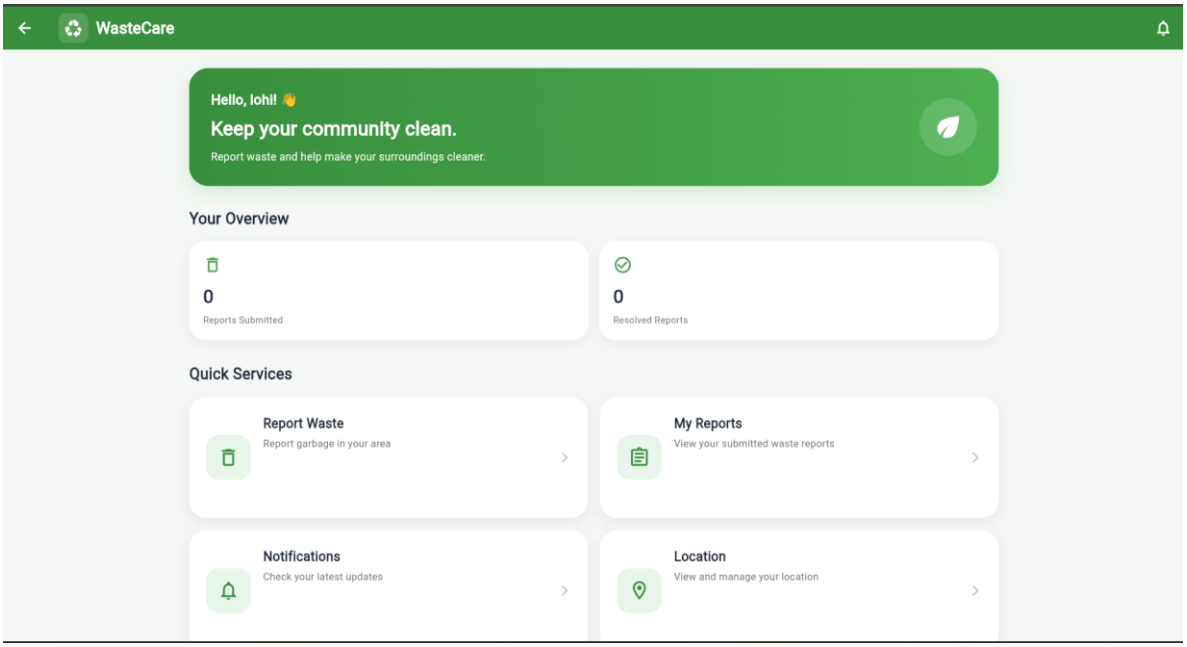
authenticated user back to login. The snippet demonstrates a direct route push; replace the example screen class with the actual class name used in the application.

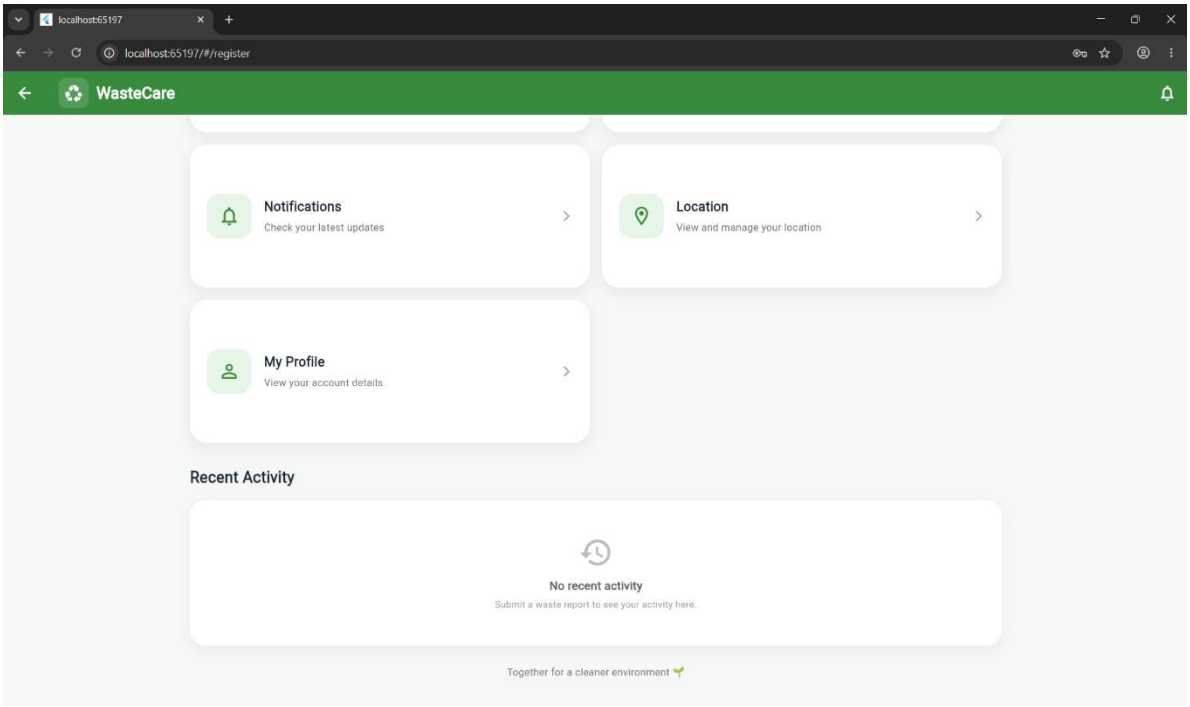
Code

```
Navigator . push (
 context ,
 MaterialPageRoute (
 builder : ( context ) = > ReportWasteScreen (
 userId : userId ,
 ) ,
 ) ,
 ) ;
```

Output

Navigator pushes a screen onto the route stack and pops it when the user returns. WasteCare uses navigation between dashboard, report form and report details.





Experiment 4(b): Implement navigation with named routes Aim

To implement navigation with named routes using examples relevant to the WasteCare project.

Description

Named routes assign a string identifier to each destination in MaterialApp and allow navigation code to refer to that identifier. This can make a multi-screen application easier to organize when routes are centrally configured. A route such as /report could represent the road damage form, while /home could represent the dashboard. The example is a syllabus demonstration and should only be described as part of WasteCare if these routes are configured in its actual source. Otherwise, document the Navigator-based navigation that the project really uses and label this as a separate experiment. The example is a syllabus demonstration and should only be described as part of WasteCare if these routes are configured in its actual source. Otherwise, document the Navigator-based navigation that the project really uses and label this as a separate experiment.

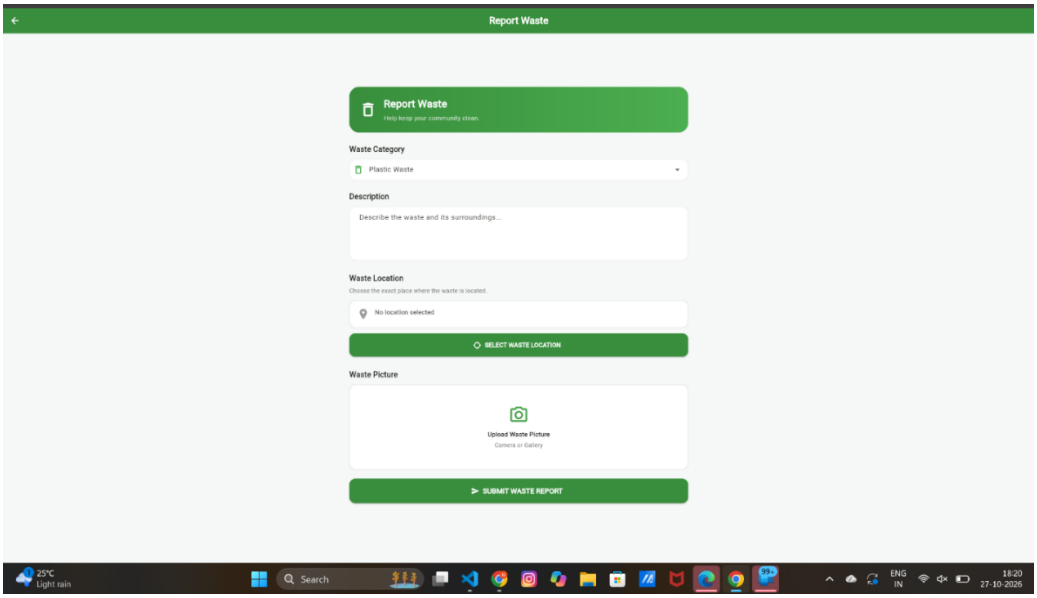
Code

```
MaterialApp (
initialRoute : '/ ' ,
routes : {
'/ ': ( context ) = > const
  WelcomeScreen () ,
'/ login ': ( context ) = >
  const LoginScreen () ,
'/ register ': ( context )
  = > const
  RegisterScreen () ,
'/ admin - login ': (
  context ) = >
const
  AdminLoginScreen () ,
} ,
) ;

// Example navigation :
Navigator . pushNamed ( context , '/ login ') ;
```

Output

Named routes centralize screen destinations. Use this pattern only if named routes are configured in the actual app; otherwise document Navigator.push as the implemented approach.



Experiment 5(a): Learn about stateful and stateless widgets

Aim

To learn about stateful and stateless widgets using examples relevant to the WasteCare project.

Description

StatelessWidget is suitable for UI that is built from supplied values and does not manage changing state internally. StatefulWidget is used when a screen needs to respond to user interaction, such as changing a selected severity, showing a loading indicator or toggling a setting. WasteCare combines static labels and reusable display components with interactive forms and controls. Keeping each widget focused on a clear responsibility improves maintainability and makes testing easier. The severity label sample demonstrates a simple stateless display component; capture the real form state for output.

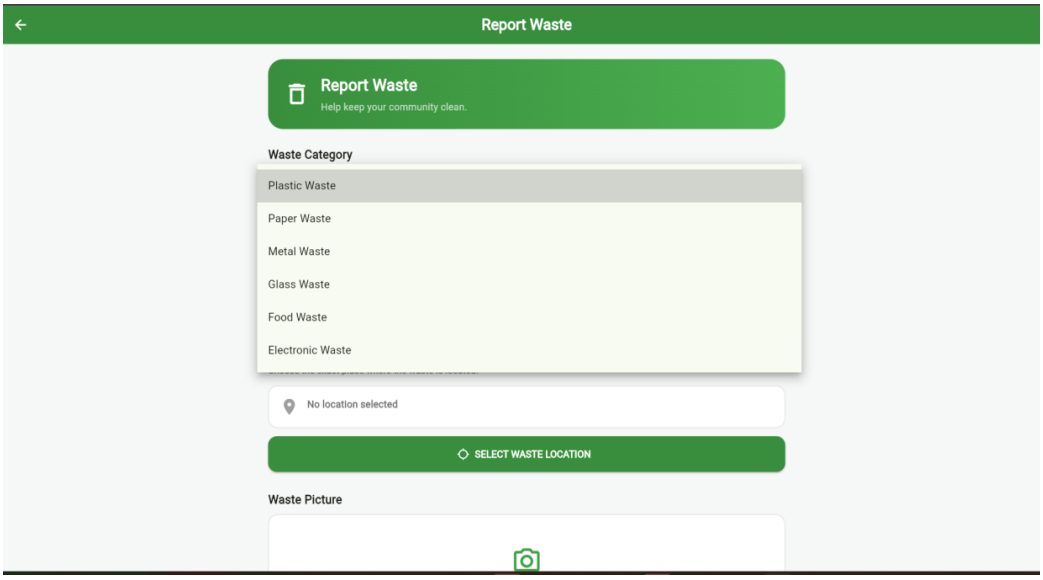
Code

```
class StatusScreen extends StatelessWidget {
const StatusScreen ({ super . key }) ;
 @override
 Widget build ( BuildContext context ) {
 final status = [
 ' Pending ' ,
 ' Assigned ' ,
 ' In Progress ' ,
 ' Completed ' ,
 ];
 return Scaffold (
 appBar : AppBar ( title : const Text ( ' Complaint Status ') ) ,

 body : ListView . builder (
 itemCount : status . length ,
 itemBuilder : ( context , index ) {
 return ListTile (
 title : Text ( status [ index ]) ,
 ) ;
 } ,
 ) ,
 ) ;
 }
 }
```

Output

StatelessWidget renders from supplied values. StatefulWidget stores changing UI state, such as selected severity, form loading or theme choice.



Experiment 5(b): Implement state management using setState and Provider

Aim

To implement state management using setstate and provider using examples relevant to the WasteCare project.

Description

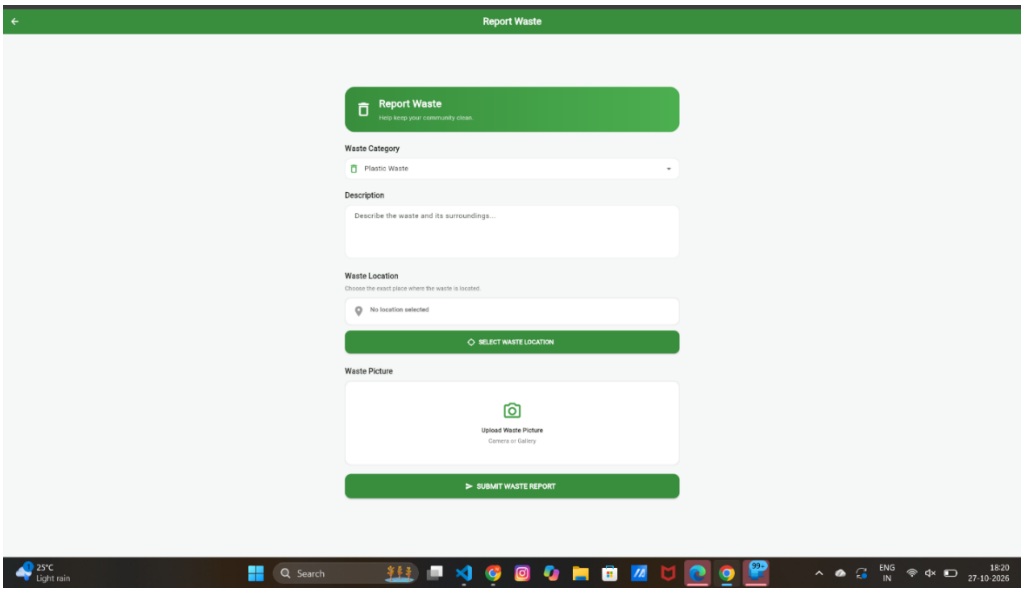
State management keeps the interface synchronized with values that change during user interaction. setState is suitable for local state within a StatefulWidget, for example updating a selected severity or switching a local option. Provider is a separate package-based approach that can expose shared state to multiple parts of an application. Provider should not be claimed as an existing RoadCare dependency unless it is present in pubspec.yaml and used in the source code. The output screenshot should show a real changing selection; if Provider is only a learning example, label it clearly as a demonstration.

Code

```
setState(() {
   selectedSeverity = 'High';
});
// Provider example for a lab demonstration only:
// context.read<ReportState>().setSeverity('High');
```

Output

setState updates local widget state. Provider is an optional shared-state approach; do not claim Provider is used in WasteCare unless it exists in pubspec and source code.



Experiment 6(a): Create custom widgets for specific UI elements

Aim

To create custom widgets for specific ui elements using examples relevant to the WasteCare project.

Description

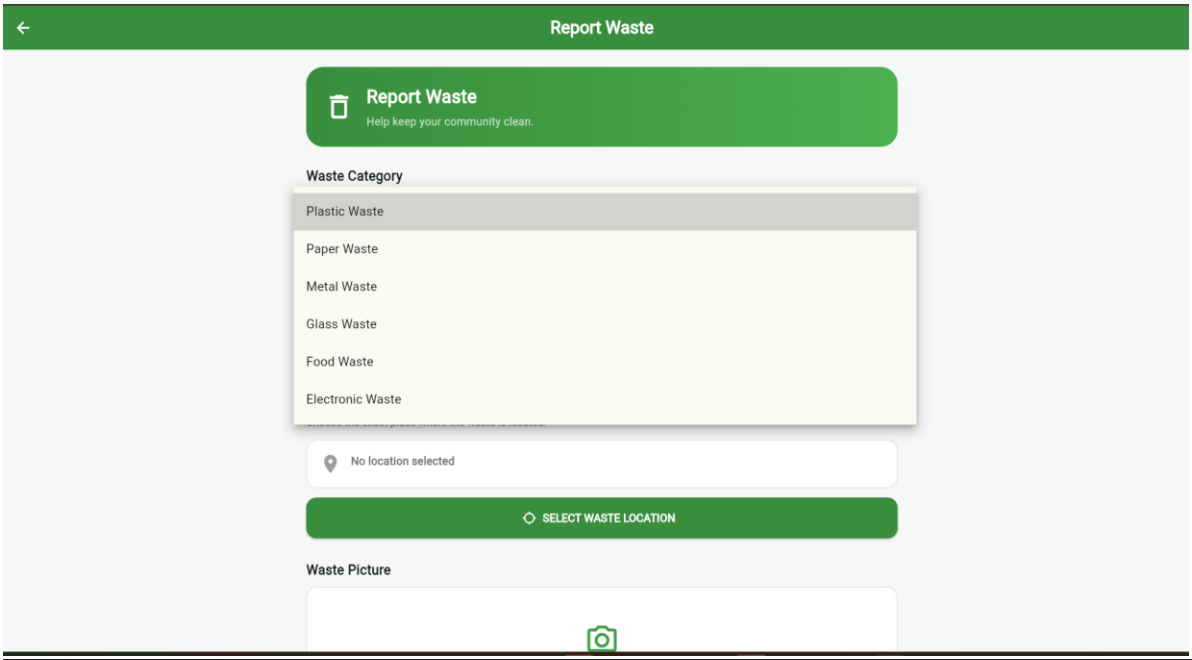
Custom widgets wrap repeated UI patterns into reusable components. A StatusChip can display values such as Pending, In Progress, Resolved or Rejected in a consistent form across report lists and details. Similar reusable widgets can represent severity labels, dashboard tiles and report cards. Reuse reduces duplicated layout code and makes later styling changes easier because a shared component can be updated in one place. The sample should be adapted to the actual WasteCare model and status values, and the screenshot should show the corresponding real UI element.

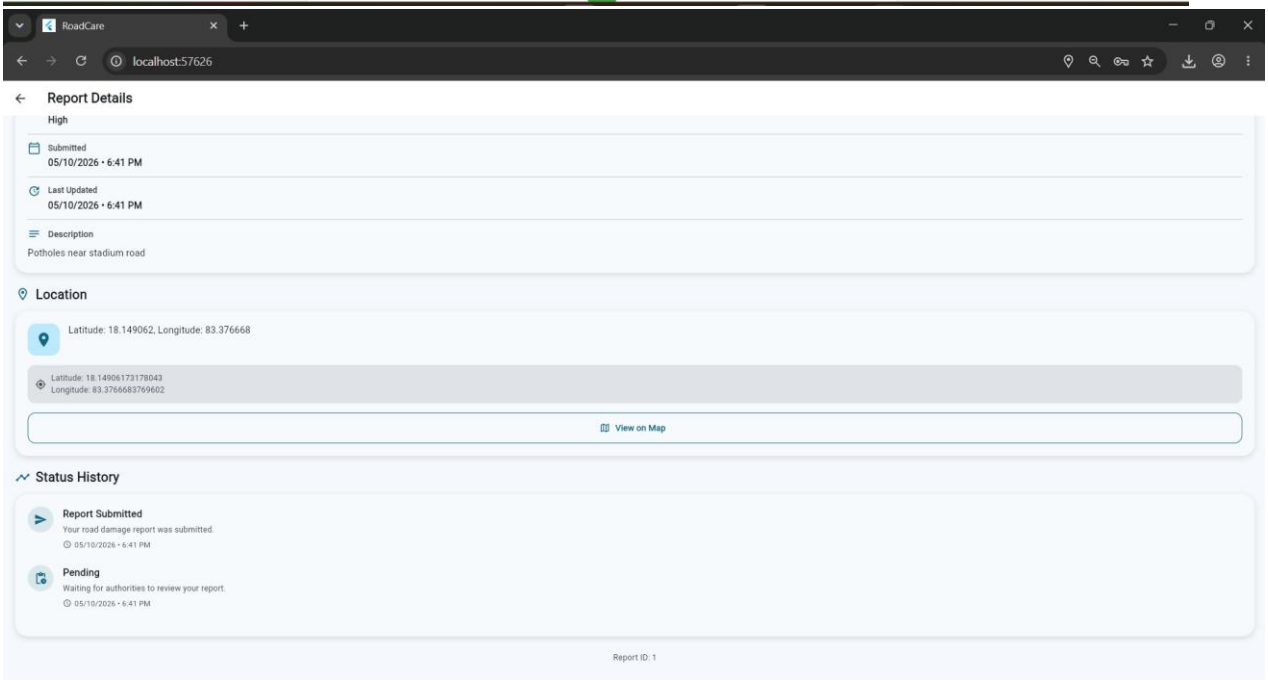
Code

```
class StatusChip extends StatelessWidget {
   const StatusChip({super.key, required this.status}); final String status;
   @override
   Widget build(BuildContext context) => Chip( label: Text(status),
   );
}
```

Output

Reusable widgets help keep report cards, severity labels, status chips and dashboard tiles consistent across screens.





Experiment 6(b): Apply styling using themes and custom styles Aim

To apply styling using themes and custom styles using examples relevant to the WasteCare project.

Description

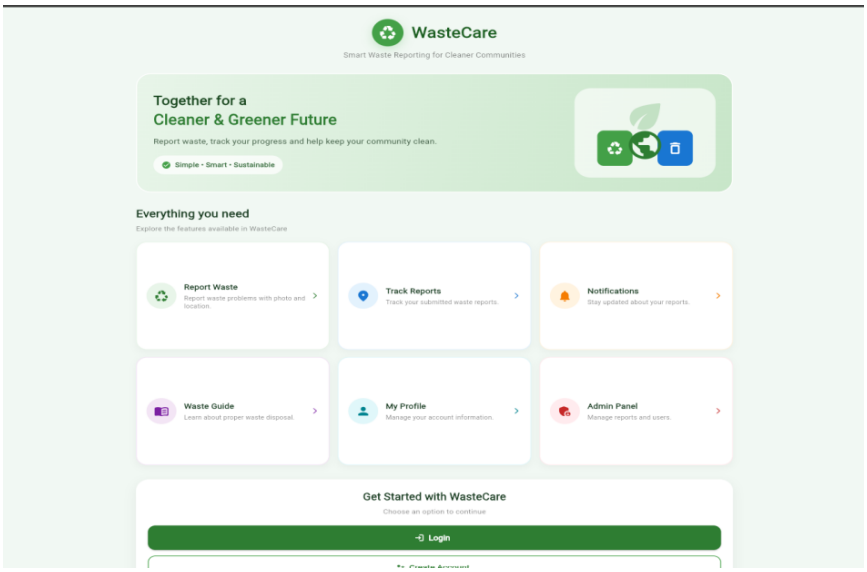
ThemeData defines common colors, typography and component styling for a Flutter application. A consistent theme helps WasteCare maintain a recognizable visual identity across the login page, dashboard, forms and admin screens. ColorScheme.fromSeed can create a coordinated palette from a primary color, while Material 3 provides modern component defaults. Individual widgets can still use custom decoration when needed, but repeated styles should be centralized where practical. The sample is a theme configuration example; make sure it matches the theme configured in the final application before submitting.

Code

```
MaterialApp( theme:
   ThemeData(
      useMaterial3: true, colorScheme:
      ColorScheme.fromSeed(
         seedColor: const Color(0xFF1565C0), ), ),
   home: const SplashScreen(),
)
```

Output

ThemeData centralizes colors, typography and component styles. WasteCare can use its blue safety-oriented palette consistently across pages.



Experiment 7(a): Design a form with various input fields

Aim

To design a form with various input fields using examples relevant to the WasteCare project.

Description

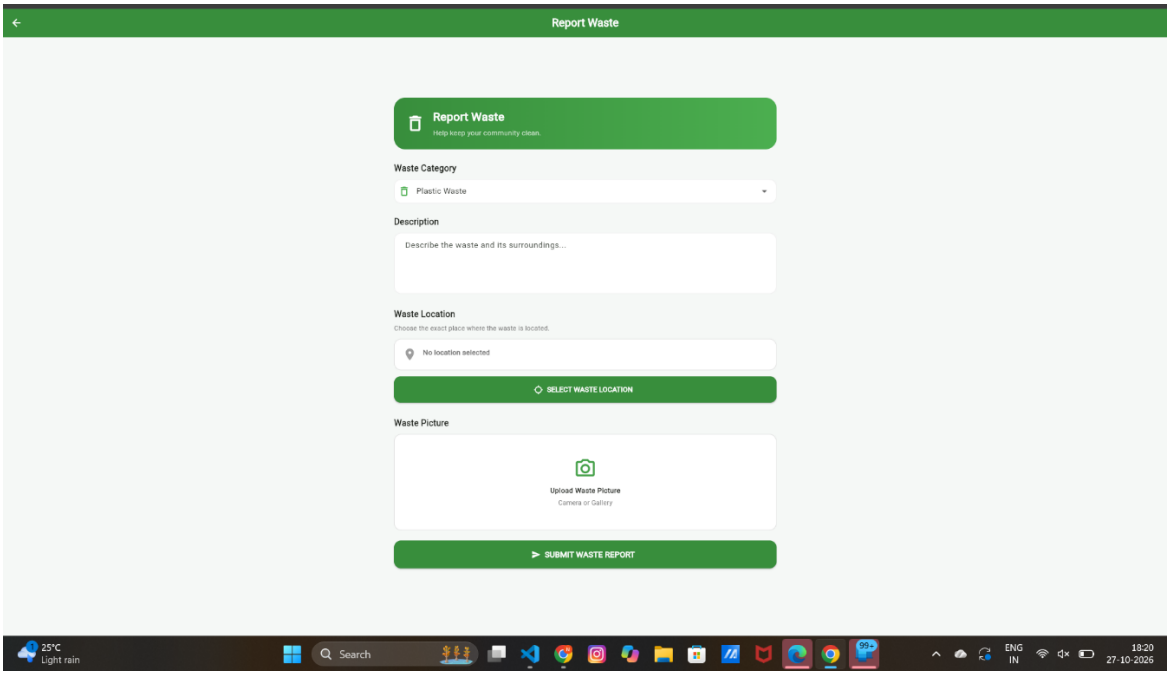
A report form collects the information needed to describe a road problem. Typical fields include a short title, a detailed description, severity, location information and optional image evidence. TextEditingController can read text input, while a Form and GlobalKey<FormState> help organize validation. Capture the actual WasteCare report form so the screenshot reflects the controls that are implemented rather than an imagined interface.

Code

```
final formKey = GlobalKey<FormState>(); final titleController
= TextEditingController(); // Inside build():
Form( key: formKey, child: TextFormField(
   controller: titleController, decoration: const
   InputDecoration(
         labelText: 'Report title',
      ), validator: (v) => (v == null || v.trim().isEmpty)
         ? 'Enter a title' : null,
   ),
)
```

Output

The road damage form gathers title, description, severity, location and optional evidence images. Controllers and form keys should match the actual app code.



Experiment 7(b): Implement form validation and error handling

Aim

To implement form validation and error handling using examples relevant to the WasteCare project.

Description

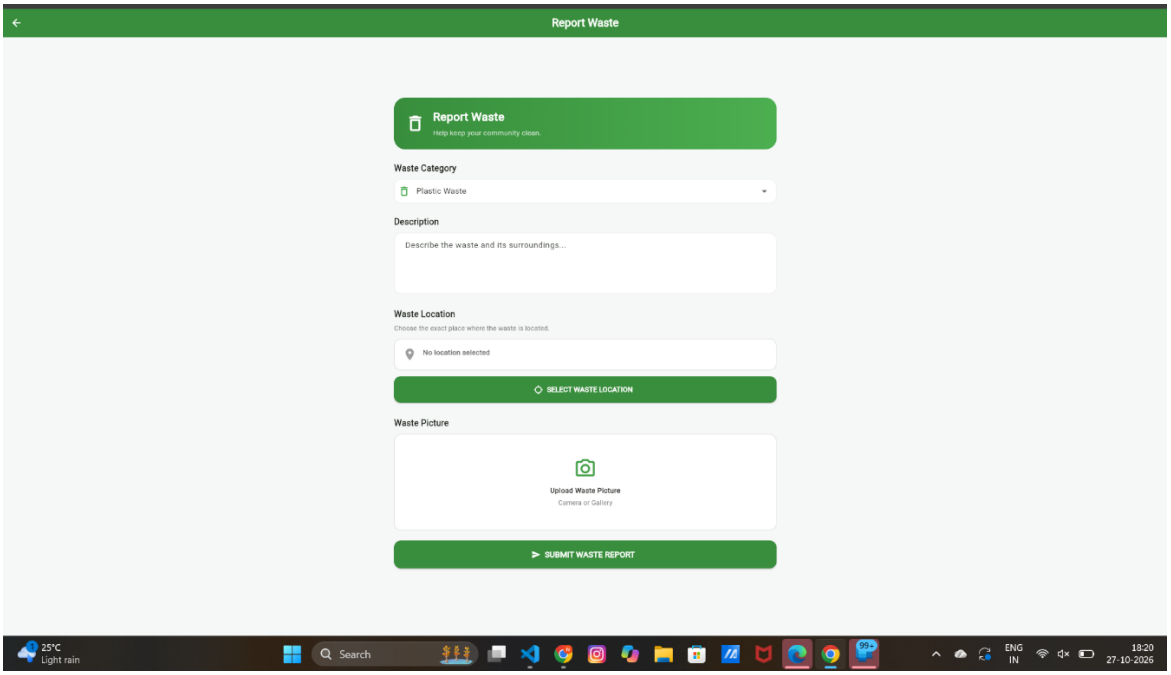
Form validation checks that required information has been provided and that entered values are suitable before saving a report. A FormField validator can return an error message when a required title or description is empty. Calling validate on the form key allows the application to decide whether to continue with submission.

Code

```
if (formKey.currentState!.validate()) {
   // Continue with the app's report-save method.
} else {
   ScaffoldMessenger.of(context).showSnackBar( const SnackBar(content:
      Text('Check required fields')),
   );
}
```

Output

Validation prevents incomplete submissions and provides understandable error messages before the report is stored.



Experiment 8(a): Add animations to UI elements using Flutter animation framework

Aim

To add animations to ui elements using flutter animation framework using examples relevant to the WasteCare project.

Description

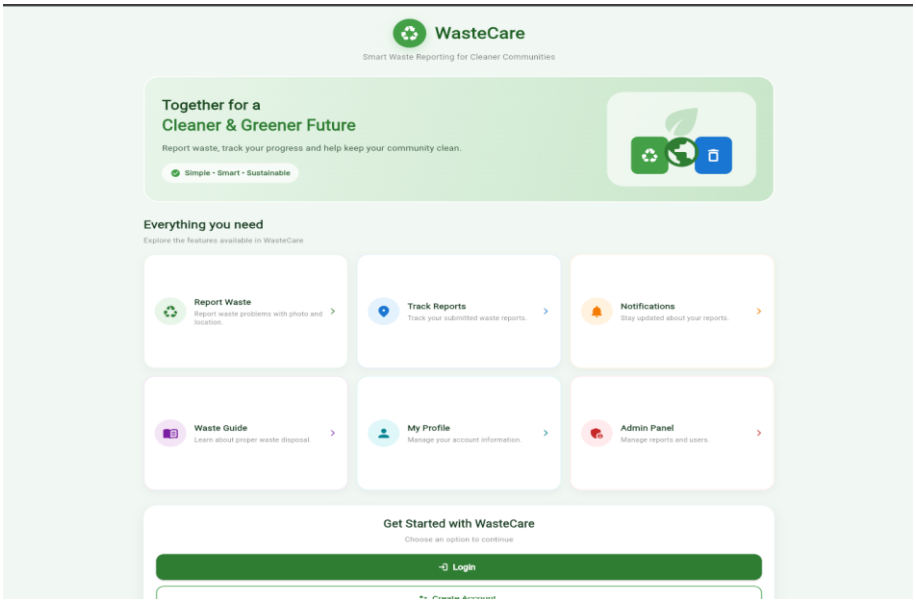
Animation can provide feedback and make screen transitions feel smoother when it is used carefully. Flutter supports implicit animations as well as explicit animations controlled by AnimationController. RoadCare may use an animated launch screen, a fade-in, a scale effect or a transition between views, depending on what is actually implemented.

Code

```
AnimatedContainer( duration: const Duration(milliseconds:
   300), curve: Curves.easeInOut, padding: const
   EdgeInsets.all(16), decoration: BoxDecoration(
      color: selected ? Colors.blue.shade100 : Colors.white, borderRadius:
      BorderRadius.circular(16),
   ), child: const Text('WasteCare'),
)
```

Output

Animations can guide attention during splash, transitions and feedback. Include only animations that are present in the final running build.



Experiment 8(b): Experiment with fade, slide and other animations

Aim

To experiment with fade, slide and other animations using examples relevant to the WasteCare project.

Description

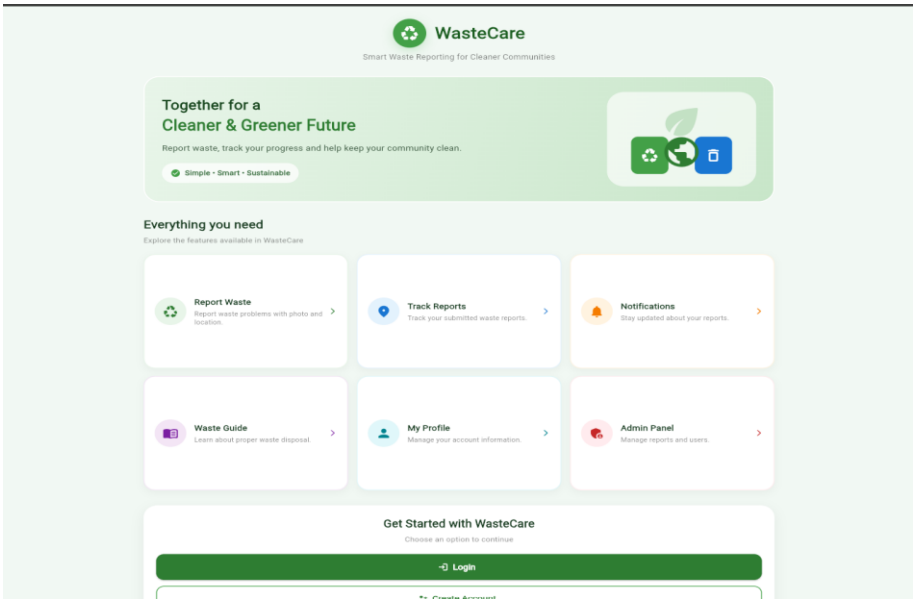
Fade transitions gradually change opacity, while slide transitions move a widget into or out of view. These effects can guide attention when opening a report or moving between sections. Flutter provides widgets such as AnimatedOpacity and SlideTransition for these patterns, with controllers required for explicit animation. Animations should remain brief, predictable and compatible with reduced-motion expectations where possible. This code is a demonstration of the animation concept unless it is already integrated in WasteCare. Use an actual screenshot or a sequence of captured frames as output evidence.

Code

```
FadeTransition( opacity: animation,
   child: const Text('RoadCare'),
)
// animation should be created and disposed by a // State using
SingleTickerProviderStateMixin.
```

Output

FadeTransition and SlideTransition animate a widget using an AnimationController. Use a real running screen recording or a representative frame as evidence.



Experiment 9(a): Fetch data from a REST API

Aim

To fetch data from a rest api using examples relevant to the WasteCare project.

Description

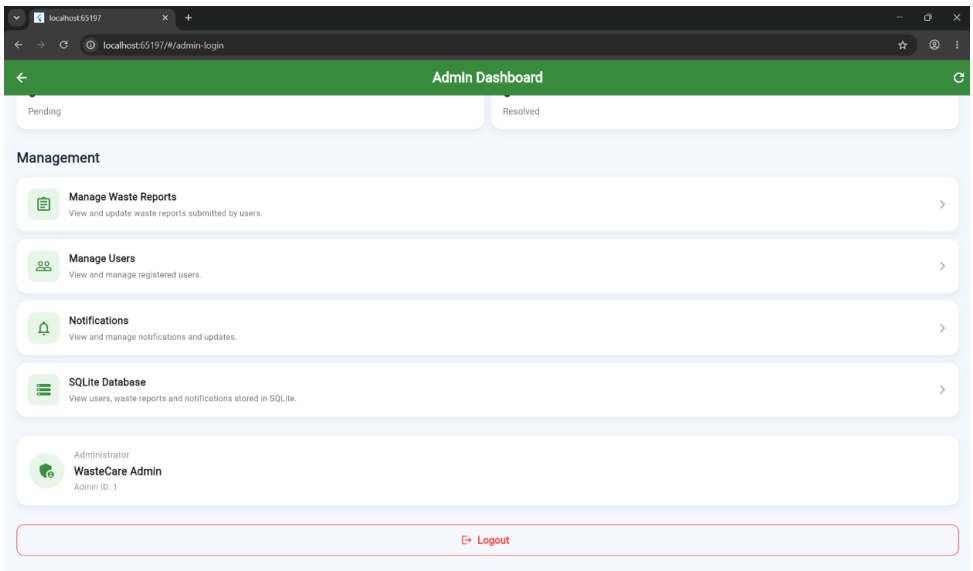
A REST API allows an application to request or submit information through HTTP endpoints. A Flutter client commonly uses a package such as http to send a request, check the response status and decode JSON. A real API demonstration should include the endpoint, request method, response handling and error states without exposing secret keys. The current RoadCare architecture uses local SQLite storage unless a remote API has been implemented separately. Therefore, keep this experiment labelled as a standalone lab demonstration if the app does not actually fetch reports from a REST service.

Code

```
// Lab demonstration only; requires http package and // a real,
approved endpoint before use.
final response = await http.get(Uri.parse(apiUrl)); if
(response.statusCode == 200) { final data =
jsonDecode(response.body);
   // Parse and display only after defining a schema.
}
```

Output

A REST client can fetch public or authority-provided road data. The known RoadCare design uses local SQLite; do not describe REST as an existing feature unless the app actually makes a network request.



Experiment 9(b): Display fetched data meaningfully in the UI Aim

To display fetched data meaningfully in the UI using examples relevant to the WasteCare project.

Description

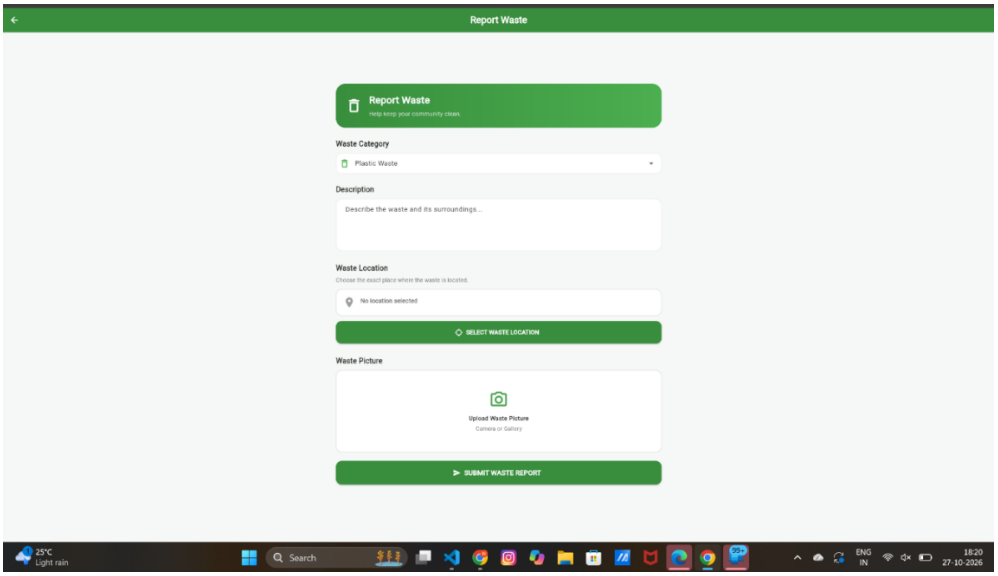
Data fetched from an API should be converted into model objects and presented in a meaningful interface rather than displayed as raw JSON. A list or card can show a title, severity, location and current status, with a loading indicator while the request is running and an error message if the request fails. The UI should also handle an empty response without showing a blank or broken screen. RoadCare's My Reports screen provides a related example of presenting structured report data from local storage. If the API is not integrated, label the API-driven code as a separate demonstration.

Code

```
ListTile( leading: const Icon(Icons.report_problem), title:
    Text(report.title), subtitle: Text(report.description),
    trailing: Text(report.status),
)
```

Output

Fetched data is parsed into model objects and displayed as list items/cards. In RoadCare, the same UI pattern can display reports read from SQLite.



Experiment 10(a): Write unit tests for UI components

Aim

To write unit tests for ui components using examples relevant to the RoadCare project.

Description

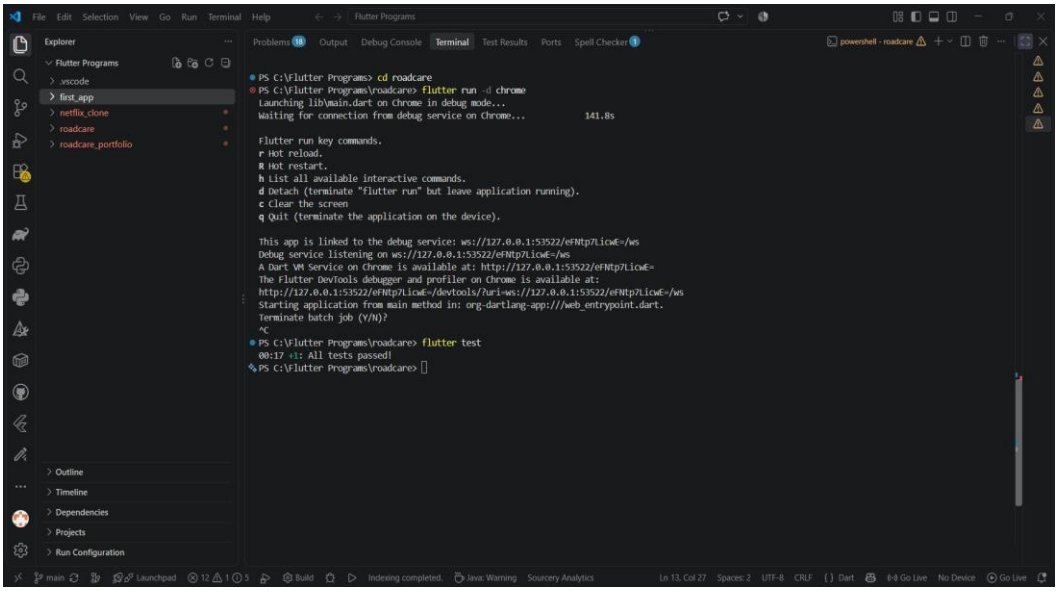
Unit and widget tests help verify that important application behavior remains correct after code changes. Unit tests can check small functions such as severity mapping or validation rules, while widget tests can verify that a form field or button appears and responds as expected. Flutter tests are usually run with the flutter test command and their real terminal output should be saved for the record. A test should contain a clear expectation rather than merely launching the app. Include only tests that exist and run successfully in the project, and label sample tests as demonstrations until they are added to the test folder.

Code

```
testWidgets('shows RoadCare title', (tester) async {
  await tester.pumpWidget(
    const MaterialApp(home: Scaffold(
      body: Text('RoadCare'),
    )), ); expect(find.text('RoadCare'), findsOneWidget);
});
```

Output

Unit/widget tests check validation, widget rendering and basic interaction. Run tests using Flutter's test runner and include actual passing test output.



Experiment 10(b): Use Flutter debugging tools to identify and fix issues

Aim

To use flutter debugging tools to identify and fix issues using examples relevant to the RoadCare project.

Description

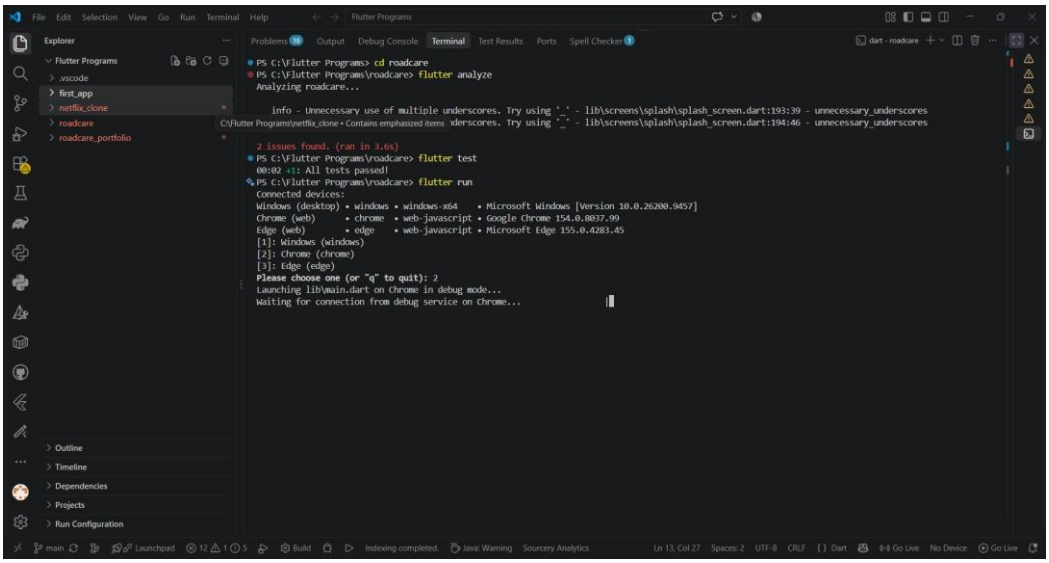
Flutter debugging tools help locate errors during development and check whether the project builds for its target platforms. flutter analyze reports static analysis issues, while debug console output can reveal runtime exceptions and failed operations. Flutter DevTools can help inspect widget layout, performance and application state when available. A useful debugging record shows the original issue, the change made and the verified result. Capture actual terminal output from the project and do not describe a warning as fixed unless the final analyzer or build output confirms it.

Code

```
flutter analyze flutter test flutter
run -d chrome flutter run -d
<device-id>
```

Output

Flutter DevTools, debug console, breakpoints and ‘flutter analyze‘ help identify layout, state and runtime issues. Record a real issue and its verified fix.



Case Study: WasteCare (waste collection System)

Introduction

WasteCare is a Flutter-based application concept for recording and managing surrounding cleaning. Users can submit a report with a description, severity, location information and image evidence. An administrator reviews submitted reports and updates their status. The application is designed to run on Android and Chrome/Web.

Problem Statement

Road defects such as potholes, cracks and damaged edges can affect road safety. A structured digital form can make reports easier to record and review. WasteCare demonstrates a local application workflow for collecting and managing this information.

Objectives

- Provide user registration and login.
- Create road damage reports with descriptive details.
- Attach image evidence where supported.
- Record location information where permissions/platform support allow it.
- Classify damage using Low, Medium, High and Critical severity.
- Allow users to view reports and their status.
- Provide administrator report review, search, filtering and status management.
- Store application records in SQLite.
- Demonstrate a responsive UI on Android and Chrome/Web.

Technology Stack

Technology	Use in WasteCare
Flutter	Cross-platform UI framework.
Dart	Application language.
SQLite	Local relational storage.
VS Code	Development and debugging.
Android SDK / emulator	Android build and testing.
Chrome	Web build and responsive layout testing.

System Architecture

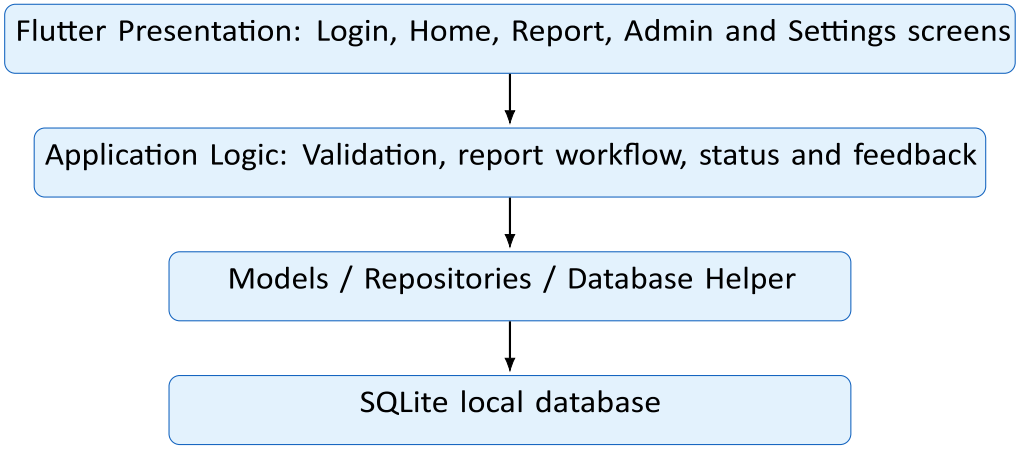


Figure 1: WasteCare high-level architecture

Database Design

The project database design includes users, reports, report images, status history, notifications and feedback. Verify the exact columns against the final `database_helper.dart` and model files before submission; the tables below describe the logical relationships, not a substitute for the actual schema.

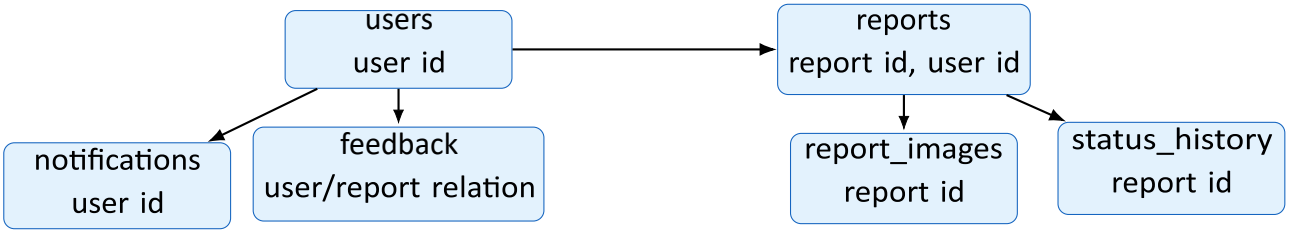


Figure 2: Logical WasteCare database relationships

users

Stores account/profile and role information.

reports

Stores report title/description, severity, status, reporter reference, location data, timestamps and archive state as applicable.

report_images

Stores image references associated with a report.

status_history

Stores report status transitions and related time information. **notifications**

Stores in-app notification records if enabled in the implemented version.

feedback

Stores feedback entries submitted through the application.

User Workflow

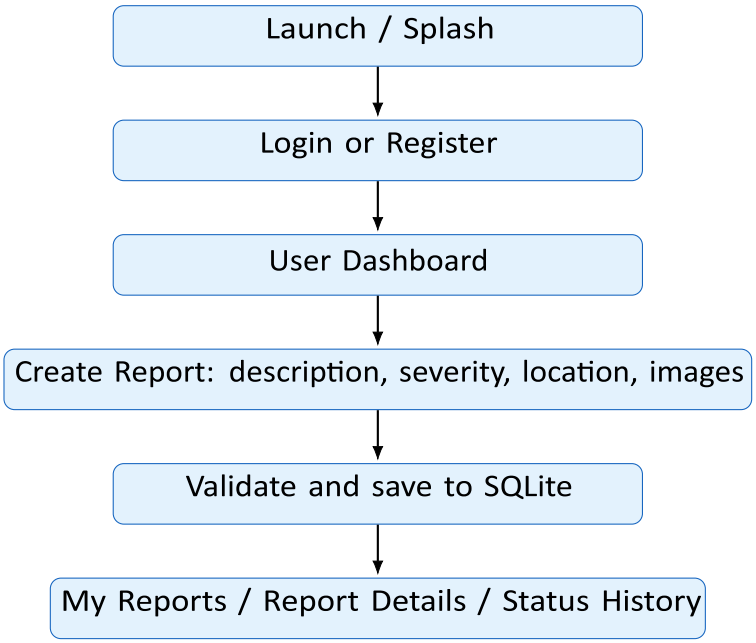


Figure 3: User report workflow

Administrator Workflow

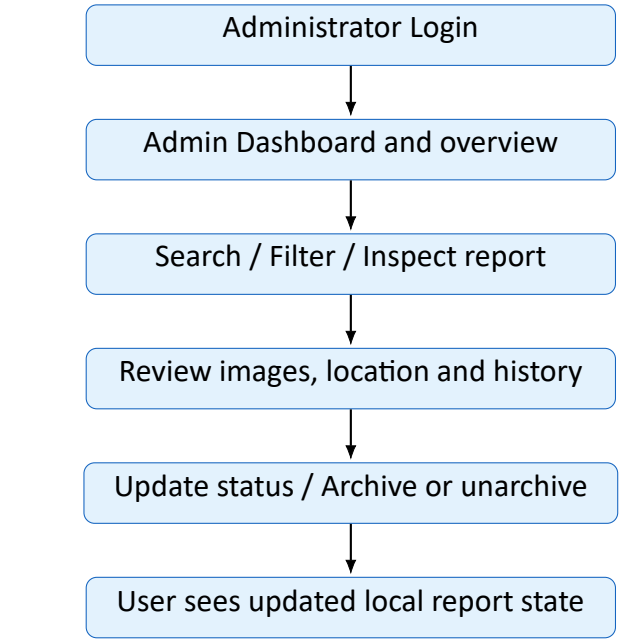
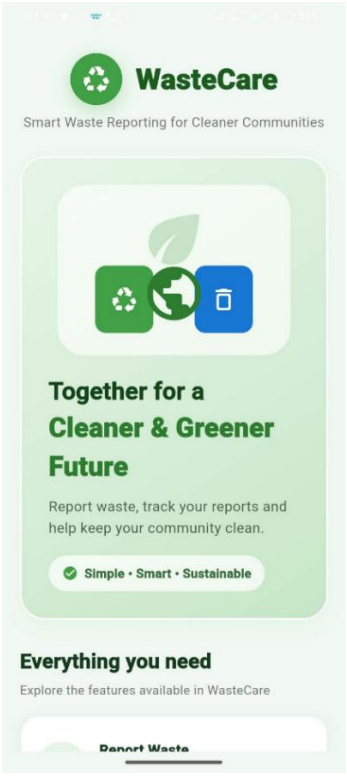


Figure 4: Administrator report management workflow

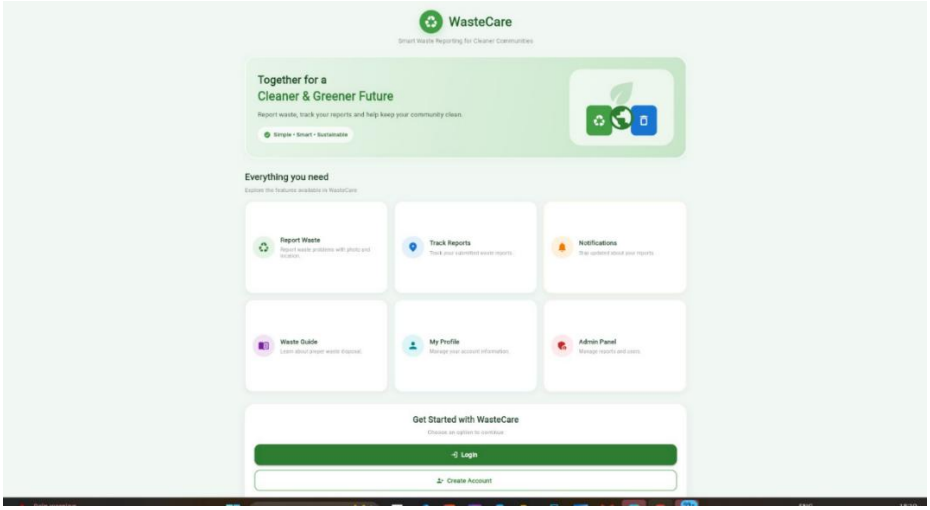
Screen-by-Screen Output Gallery

This section presents the visual outputs of the application through a collection of screenshots. Each screen demonstrates the functionality, user interface, and overall design of the system. Android and Chrome screenshots are included to illustrate the application's appearance across different platforms.

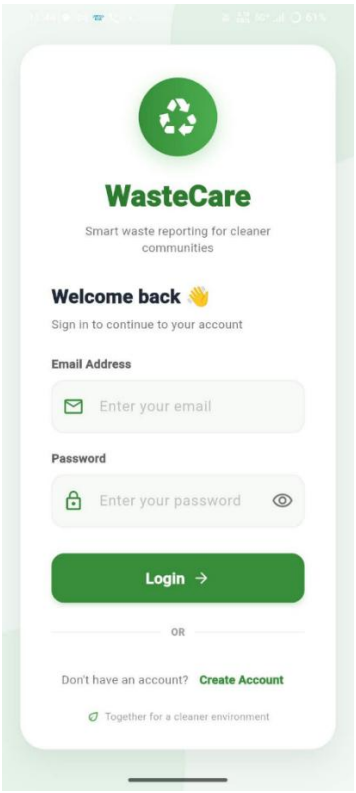
Screen 1: Splash screen on Android.



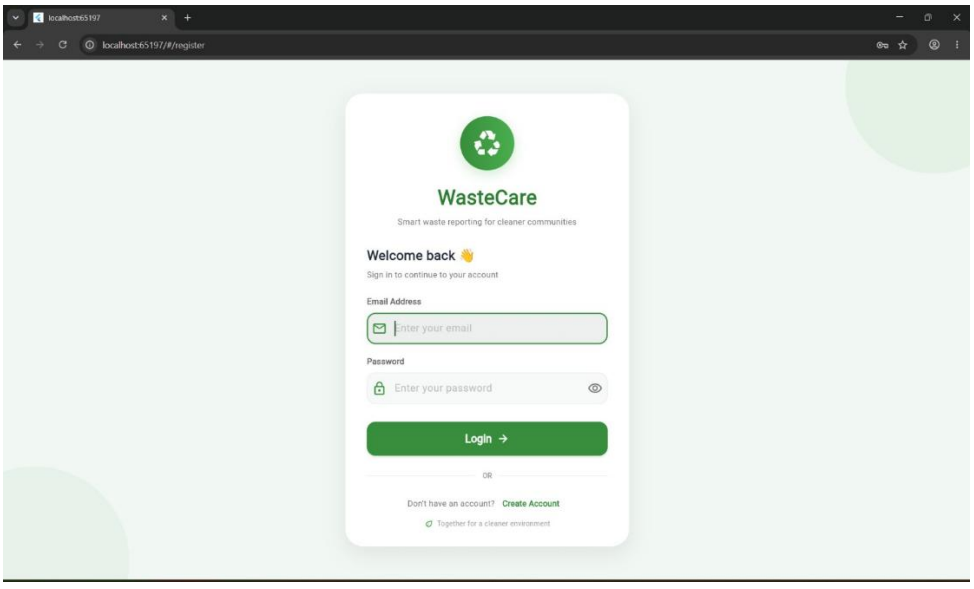
Screen 2: Splash screen on Chrome.



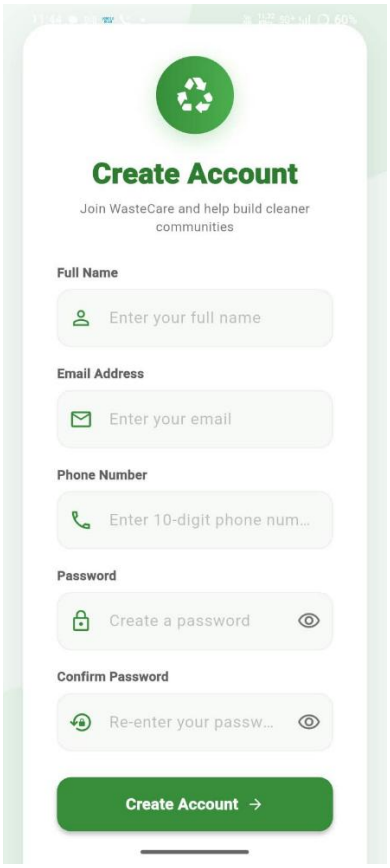
Screen 3: Login screen on Android.



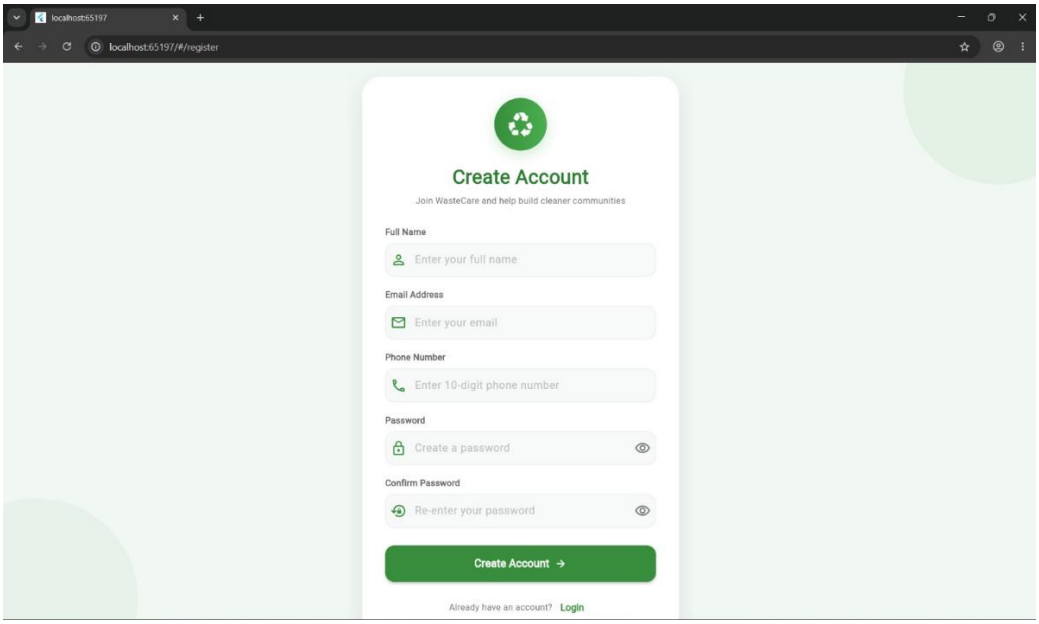
Screen 4: Login screen on Chrome.



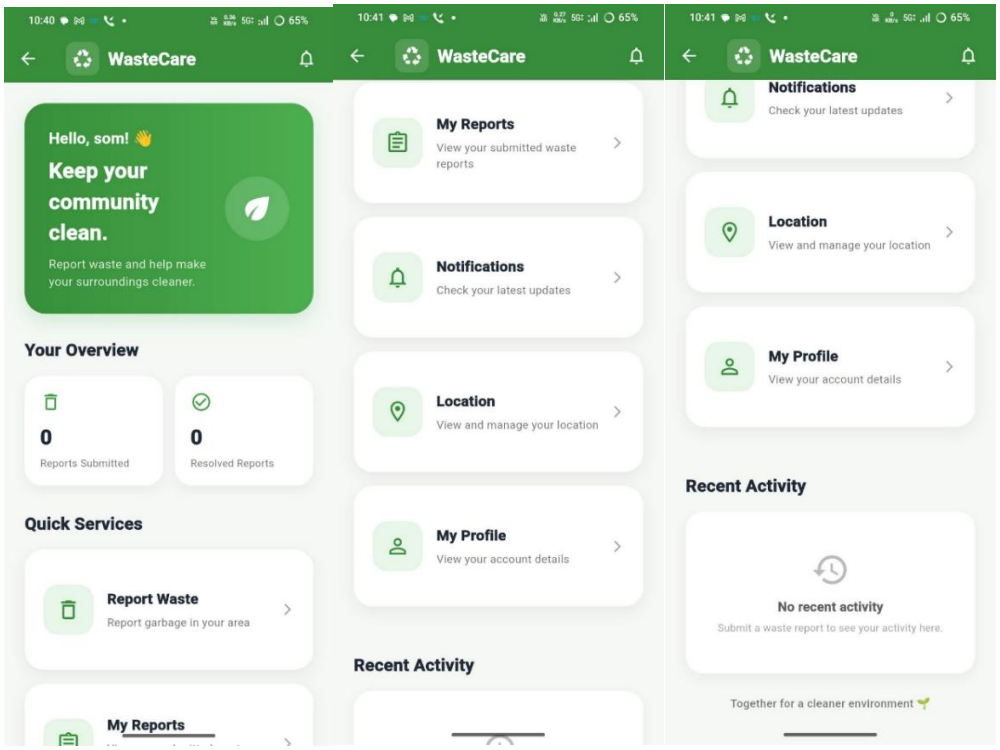
Screen 5: Registration screen on Android.



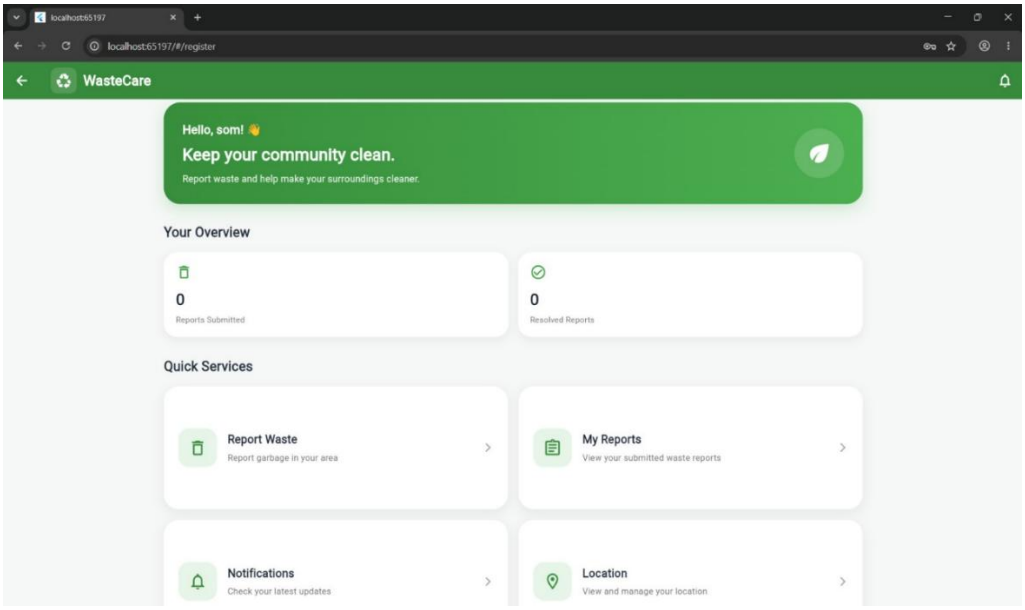
Screen 6: Registration screen on Chrome.



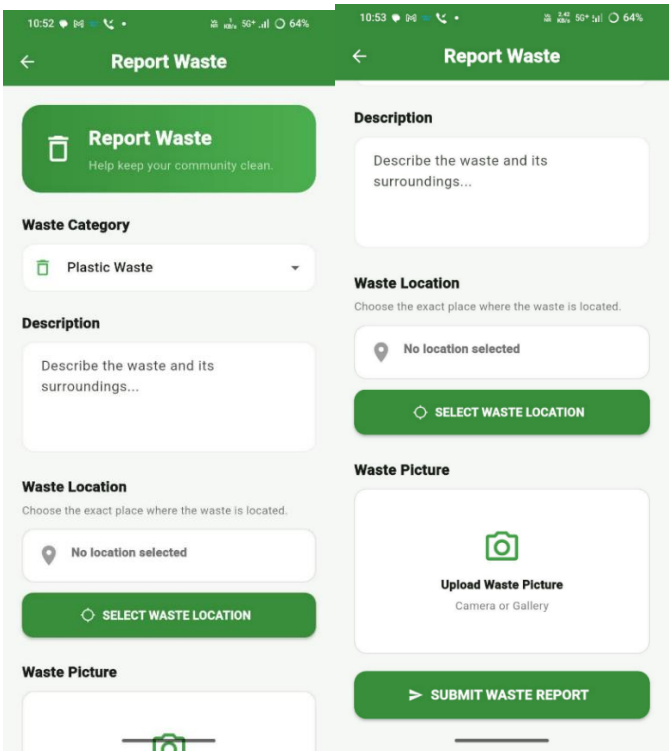
Screen 7: User dashboard on Android.



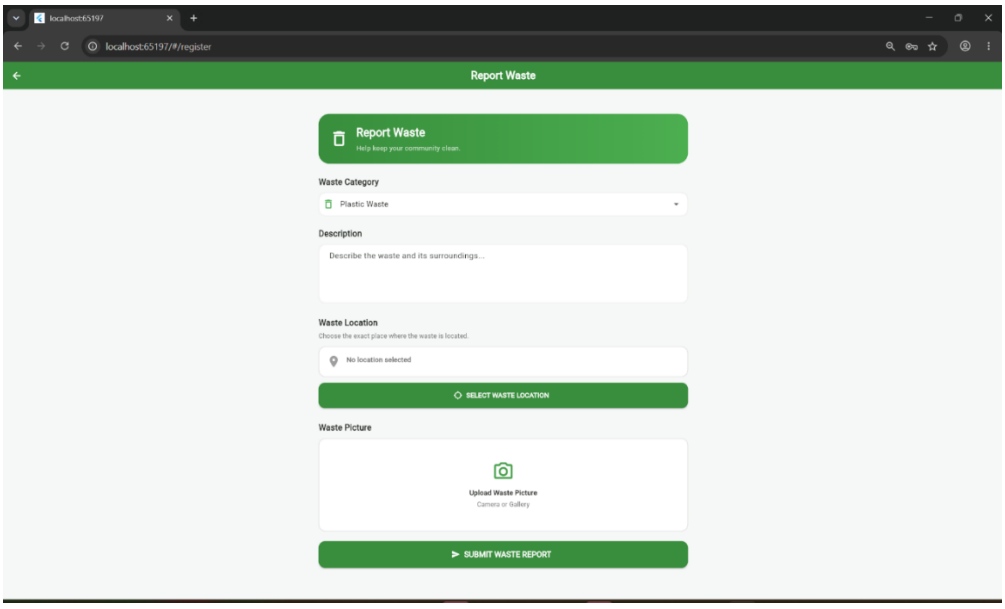
Screen 8: User dashboard on Chrome.



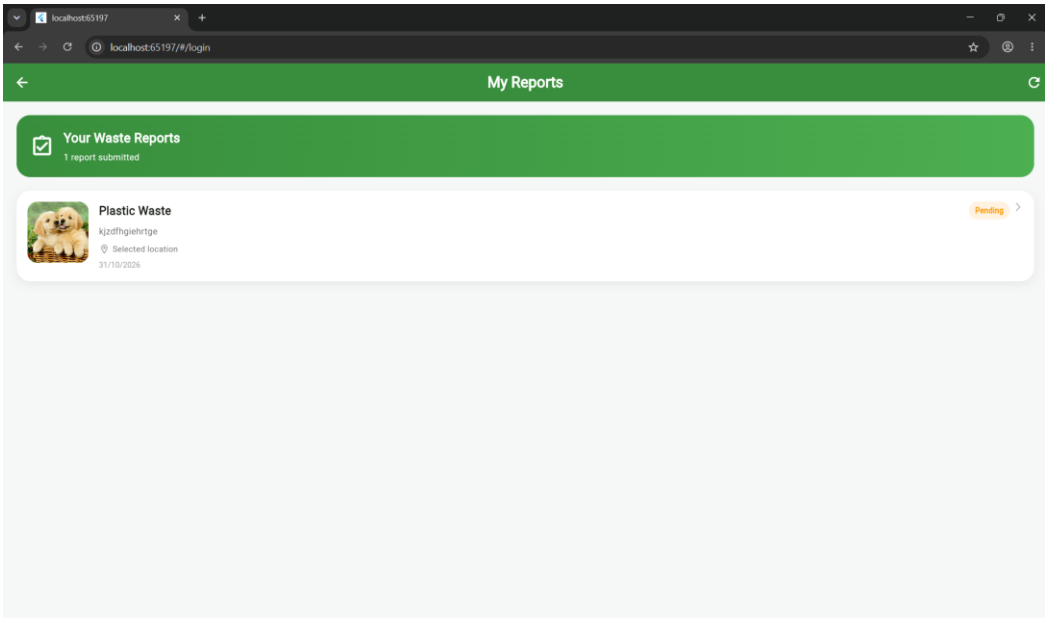
Screen 9: Road Damage Report form on Android.



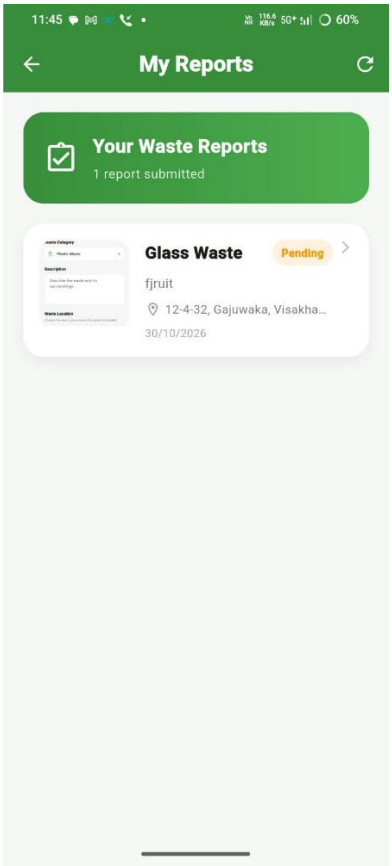
Screen 10: Road Damage Report form on Chrome.



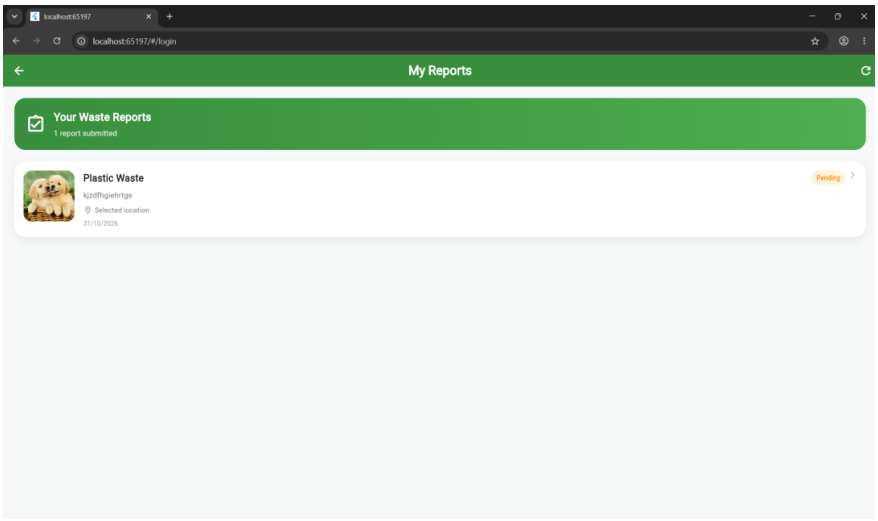
Screen 11: My Reports list on Chrome.



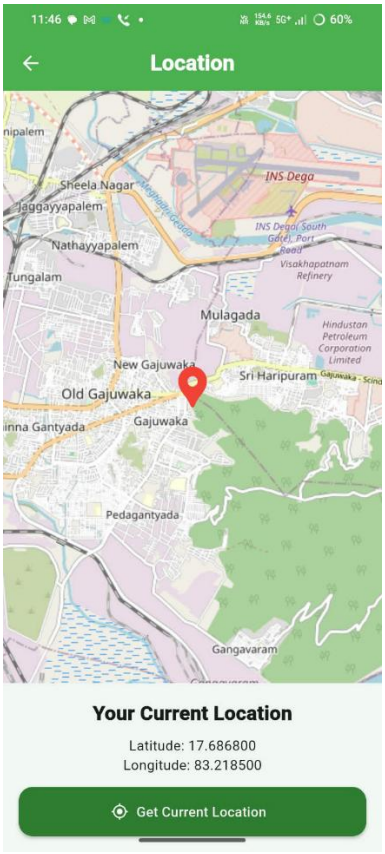
Screen 12: Report Details screen on Android.



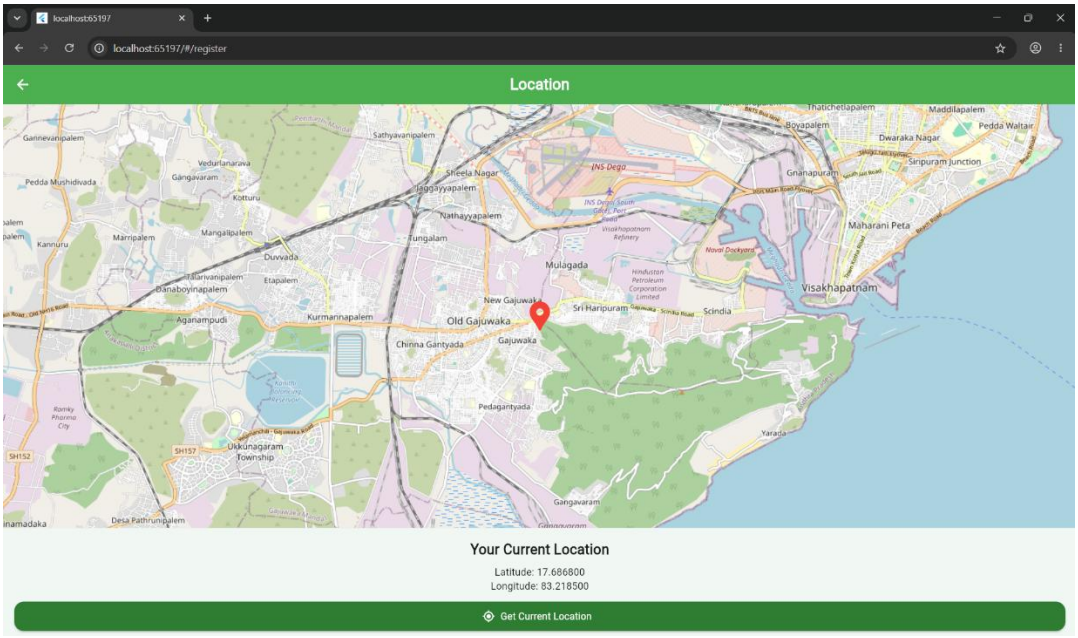
Screen 13: Report Details screen on Chrome.



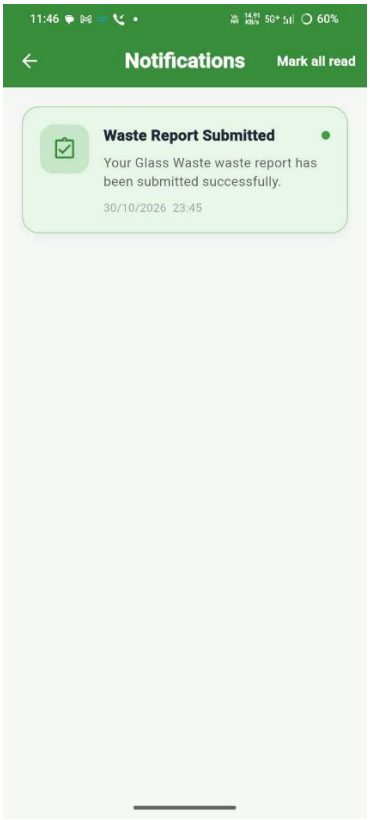
Screen 14: Report Map screen on Android.



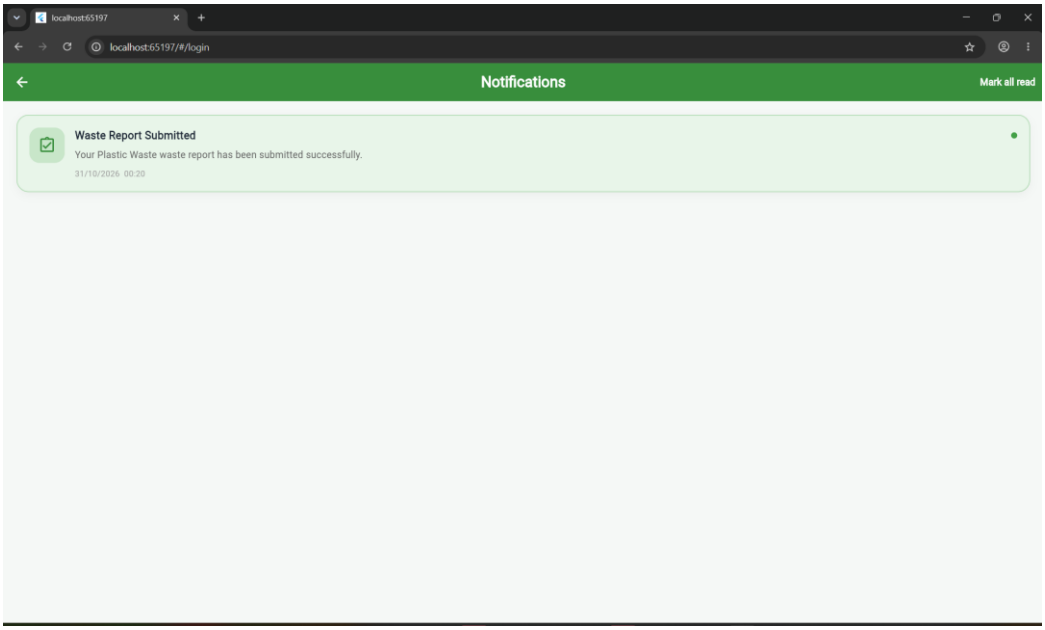
Screen 15: Report Map screen on Chrome.



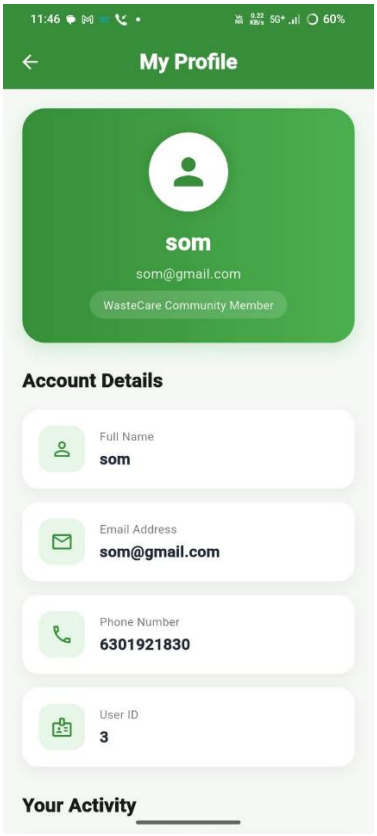
Screen 16: Notifications screen on Android.



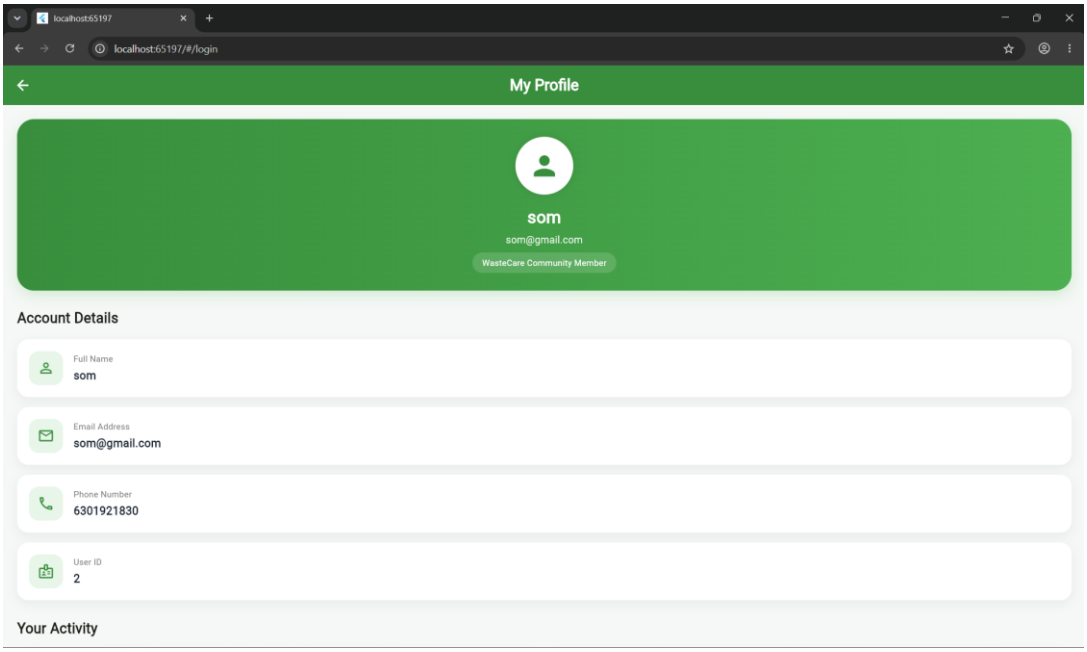
Screen 17: Notifications screen on Chrome.



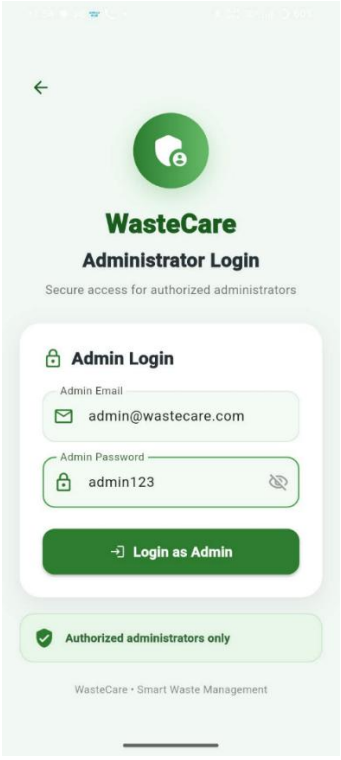
Screen 18: User Profile screen on Android.



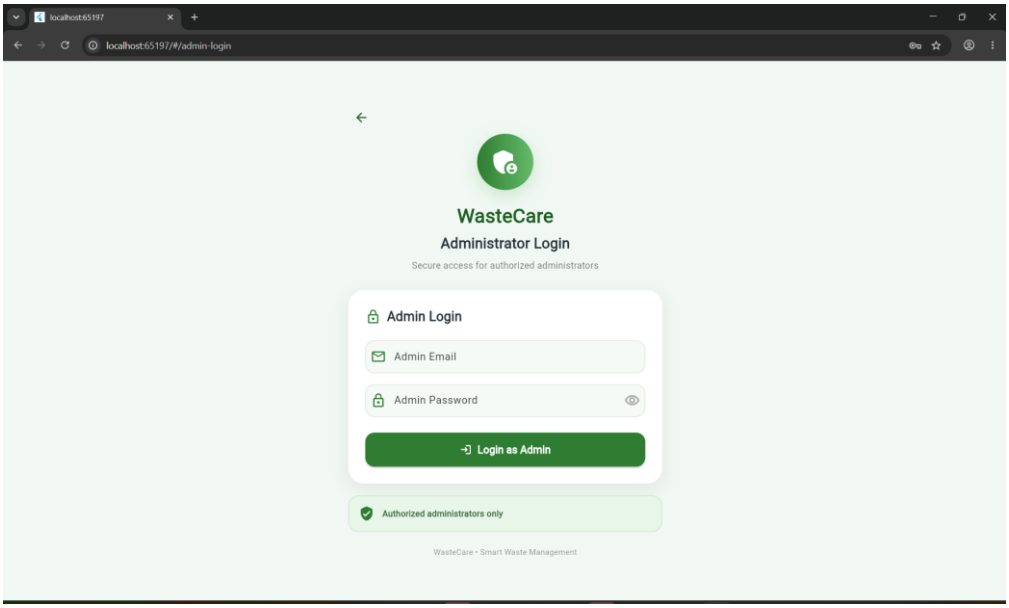
Screen 19: User Profile screen on Chrome.



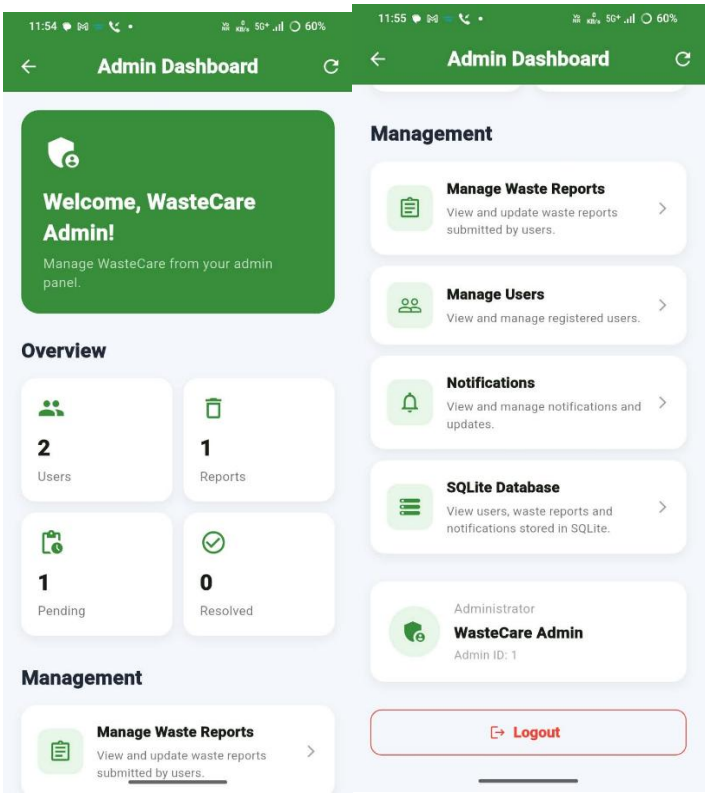
Screen 20: Administrator login screen on Android.



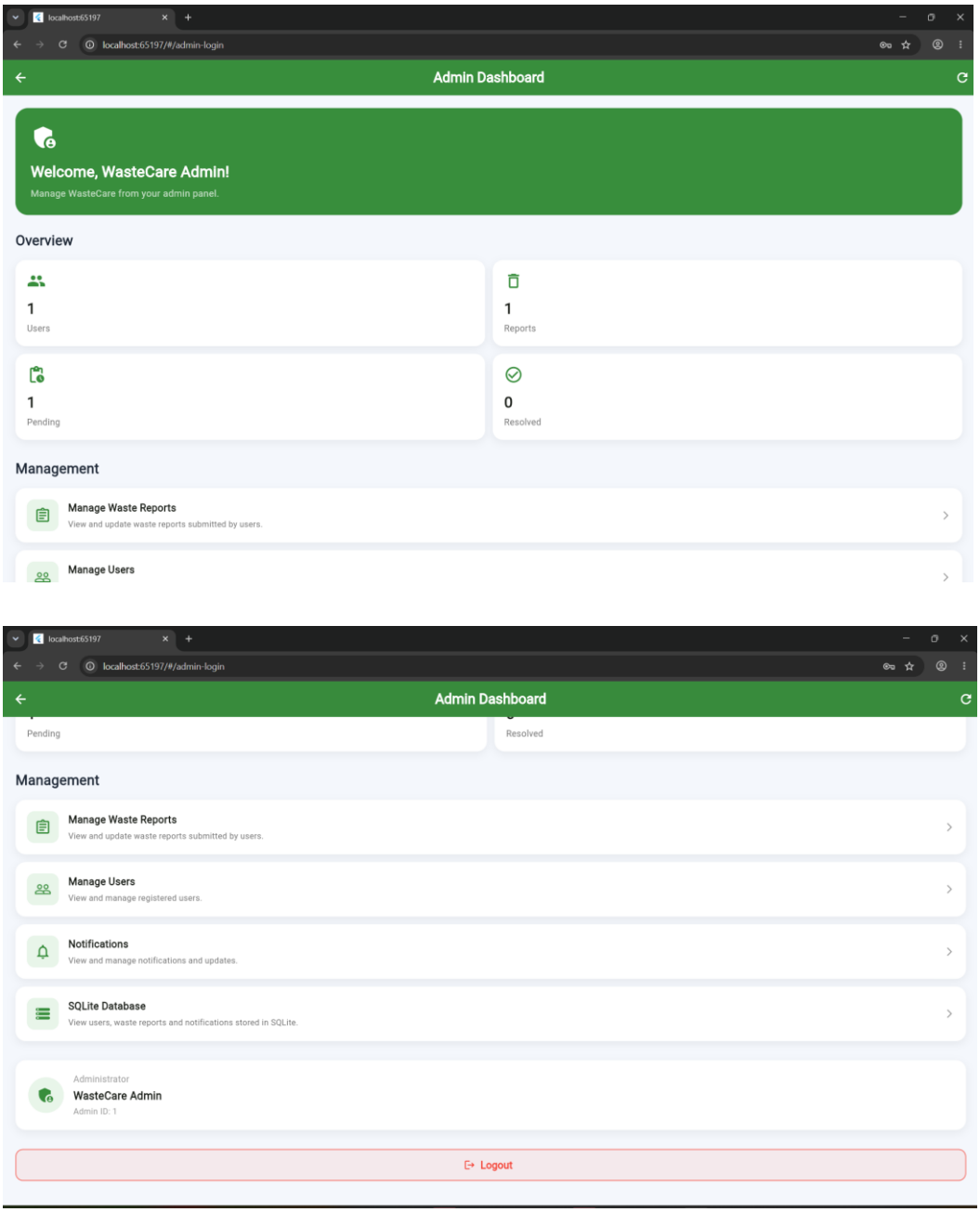
Screen 21: Administrator login screen on Chrome.



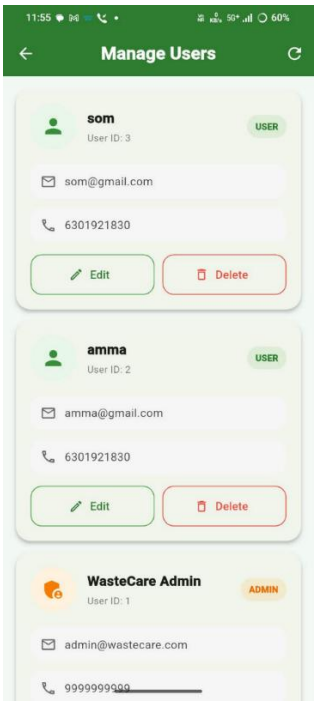
Screen 22: Administrator dashboard on Android.



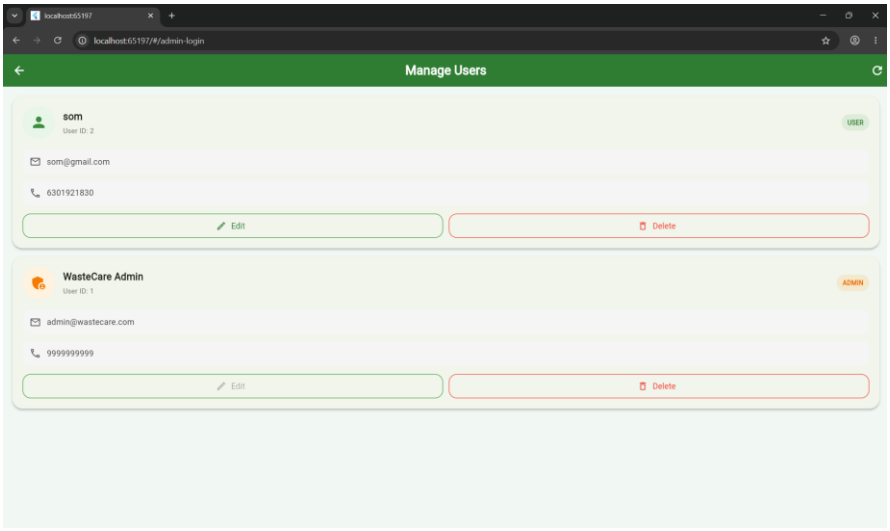
Screen 23: Administrator dashboard on Chrome.



Screen 26: Admin Manage Users



Screen 27: Admin Manage users



Android and Chrome/Web Comparison

Aspect	Android	Chrome/Web
Screen shape	Usually narrow/portrait; touch interaction.	Browser window can be narrow or wide; responsive layout should adapt.
Image selection	Uses Android-supported picker/permissions.	Uses browser file selection; behavior depends on browser and app implementation.
Location	Requires device permission and available location services.	Requires browser permission and a supported secure context.
Storage	SQLite database on the Android app/device.	Web storage/database behavior depends on the implementation; verify that the chosen SQLite package supports web. Separate platform data stores should not be assumed to sync.
Navigation	Touch/back navigation.	Mouse/touch and browser navigation behavior.

Security and Privacy Considerations

Validate form input; avoid displaying private account details unnecessarily; request platform permissions only when needed; protect administrator access; and do not store plaintext passwords in a production system. Document the exact password handling in the implemented source code. Because SQLite is local storage, data on Android and Chrome may be separate and is not automatically synchronized to other devices.

Testing Plan

ID	Test	Expected result	Evidence
T01	Register with valid values	Account creation follows app logic	registration screenshot
T02	Login with valid account	User dashboard opens	login/dashboard screenshot
T03	Submit empty report form	Validation message shown if required fields are enforced	validation screenshot

T04	Submit a valid report	Report is saved and appears in report list	success/My Reports screenshot
T05	Open report details	Correct report details are displayed	details screenshot
T06	Update report status as admin	Status reflects the update	Admin status screenshot
T07	Search/filter reports	Matching reports are displayed	Search Screenshot
T08	Open feedback screen	Feedback UI is displayed	feedback screenshot
T09	Run flutter analyze	Analyzer output is reviewed and issues addressed	terminal screenshot
T10	Run flutter test	Tests execute; record the actual result	Test output screenshot
T11	Run Android build	Core screens work on Android	Android captures
T12	Run Chrome build	Core screens work on Chrome	Chrome captures

---

Limitations

The current application should be described as a local SQLite application unless remote synchronization has actually been implemented. Live authority integration, cloud synchronization, remote push notifications, automatic AI damage detection and real-time multi-user synchronization must be listed as future work if they are not present in the current build.

Future Enhancements

Possible enhancements include an authenticated backend, secure cloud synchronization, authority integration, map-based report visualization, AI-assisted pothole detection, multilingual support, accessibility improvements, and automated test coverage.

Conclusion

WasteCare demonstrates a practical cross-platform UI design case study that connects Flutter widgets, forms, navigation, responsive layouts, image/location input, local data management and administrator workflows. The experiment record relates syllabus concepts to the project while keeping demonstration snippets distinct from verified implemented features.

References

1. Flutter Documentation, https://docs.flutter.dev/.
2. Dart Documentation, https://dart.dev/guides.

3. SQLite Documentation, https://www.sqlite.org/docs.html.
4. FixMyStreet Citizens Guide, https://fixmystreet.org/training/citizens/.
5. SeeClickFix application listing, https://play.google.com/store/apps/details?id=com.seeclickfix.ma.android.
6. Google Maps Help, https://support.google.com/maps/answer/3273406.
7. Waze hazard reporting help, https://support.google.com/waze/answer/13739290.